

GraphQL

From the purpose of API query languages and your very first query, through the type system and execution engine, to security, performance and federated production architectures.



ORIENTATION

What this tutorial is

A single, continuous course. Each section builds on the one before it, and every concept is introduced with syntax, a worked example, and the reasoning behind it.



Who it is for

Engineers, data scientists and architects who consume or build APIs. No prior GraphQL knowledge is assumed; comfort with JSON and HTTP helps.



How it is built

18 sections, from why query languages exist to federated production graphs. Every section ends with a checkpoint to test yourself.



How to use it

Type the examples into a GraphQL IDE as you go. Syntax sticks when you see the server validate, reject and answer your own queries.

ORIENTATION

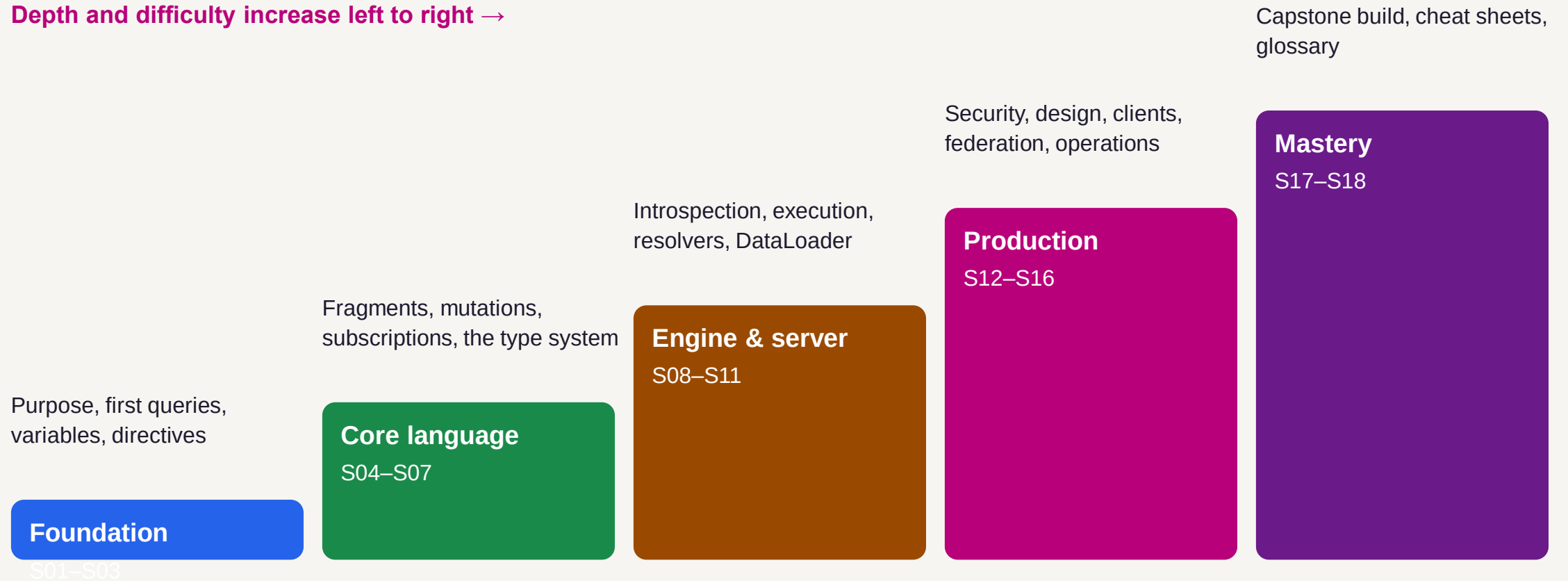
The route map: eighteen sections, five levels

01 Why API query languages exist Foundation	02 Your first queries Foundation	03 Variables & directives Foundation
04 Fragments Core	05 Mutations Core	06 Subscriptions Core
07 The type system & SDL Core	08 Introspection & tooling Intermediate	09 Execution & resolvers Intermediate
10 Building a server Intermediate	11 Performance & DataLoader Intermediate	12 Security Advanced
13 Pagination & schema design Advanced	14 Clients & caching Advanced	15 Federation & advanced architecture Advanced
16 Testing & operations Advanced	17 Capstone project Mastery	18 Reference & wrap-up Mastery

ORIENTATION

The learning path climbs from syntax to systems

Depth and difficulty increase left to right →



ORIENTATION

How to read the code slides in this deck

```
legend.graphql
# A comment explains intent
query GetBook($id: ID!) {
  book(id: $id) {
    title
    author @include(if: true) {
      name
    }
  }
}
```

1

1. Comments

Grey text after # is ignored by the server.

3

2. Keywords & variables

Pink = keywords such as query. Green = \$variables.

4

3. Type names

Blue = type names such as ID; ! means non-null.

4. Directives

Orange = directives such as @include(if: ...).

ORIENTATION

What you need before starting



HTTP & JSON

Know what a request, response, status code and header are, and be able to read nested JSON objects and arrays.



One programming language

Server examples use JavaScript/TypeScript and Python. The ideas transfer to Java, Go, C#, Rust and Ruby.



A GraphQL IDE

GraphiQL, Apollo Sandbox, Altair or Insomnia. Each gives autocomplete and validation driven by the schema.



A test API

Any public GraphQL endpoint, or the bookstore server built in Section 10 of this tutorial.

GOOD NEWS

Every concept in the first seven sections can be learned with nothing but an IDE pointed at a GraphQL endpoint. Servers come later.

ORIENTATION

By the end you will be able to...

- 01** Explain what problem API query languages solve and when GraphQL is — and is not — the right tool.
- 02** Read and write queries, mutations and subscriptions using variables, aliases, fragments and directives.
- 03** Design a schema in SDL with scalars, objects, enums, inputs, interfaces, unions and custom directives.
- 04** Explain exactly how a server parses, validates and executes a document, including null propagation.
- 05** Build a production-grade server with context, auth, DataLoader batching and error handling.
- 06** Protect a graph with depth limits, cost analysis, trusted documents and field-level authorisation.
- 07** Design cursor pagination, mutation payloads and evolvable schemas without versioning.
- 08** Reason about client caches, federation, incremental delivery and operating GraphQL in production.

SECTION 01 · FOUNDATION

Why API query languages exist

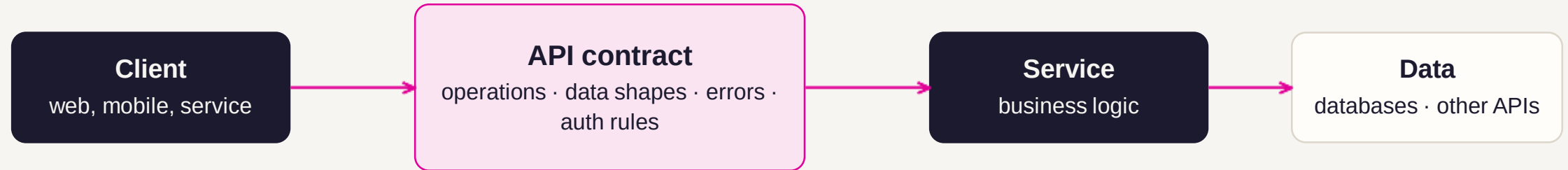
Before learning any syntax, understand the problem. Every GraphQL feature is an answer to a concrete pain in how clients fetch data.

IN THIS SECTION

- 01 What an API really is
- 02 The data-fetching problem
- 03 Over-fetching and under-fetching
- 04 Round trips and waterfalls
- 05 What a query language is
- 06 A short history
- 07 GraphQL at a glance
- 08 REST vs GraphQL vs gRPC
- 09 When not to use GraphQL

PURPOSE

An API is a contract between a consumer and a provider



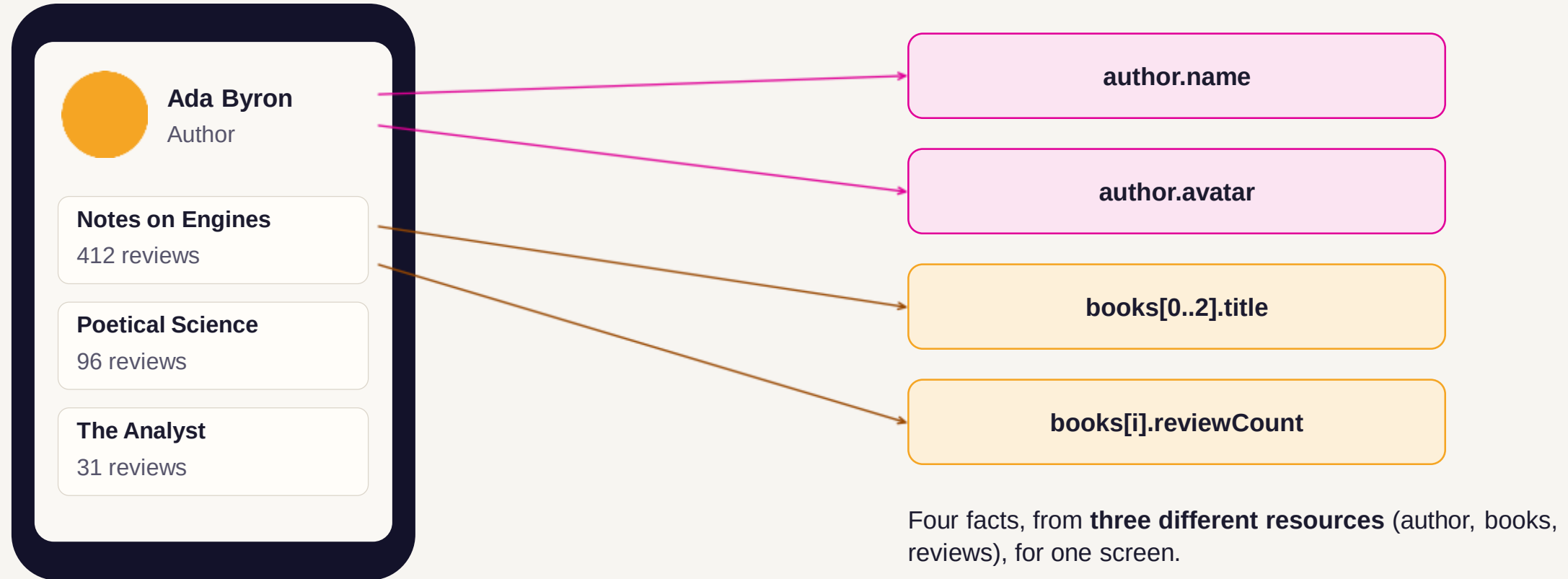
An **API** (Application Programming Interface) is a contract: it defines which operations a consumer may request, what shape the inputs and outputs take, and how failures are reported. The consumer depends only on the contract — never on the implementation behind it.

Why the contract matters: teams on either side can change independently as long as the contract holds.

The open question: who decides the shape of the data — the server that owns it, or the client that displays it?

THE DATA-FETCHING PROBLEM

A single screen needs a precise slice of connected data



THE DATA-FETCHING PROBLEM

Resource-oriented APIs make the client assemble the tree



5 HTTP round trips before the screen can render.

Each response carries fields the screen never shows.

OVER-FETCHING

Over-fetching: the server decides the shape, the client pays for it

GET /authors/42 → response

```
{
  "id": 42,
  "name": "Ada Byron",
  "avatarUrl": "https://.../ada.png",
  "email": "ada@example.com",
  "bio": "Mathematician and writer...",
  "birthDate": "1815-12-10",
  "createdAt": "2024-01-02T09:00:00Z",
  "updatedAt": "2026-05-11T14:21:07Z",
  "social": { "site": "...", "handle": "..."},
  "stats": { "followers": 1204, "books": 3 },
  "settings": { "locale": "en-GB", ... }
}
```

WASTED PAYLOAD

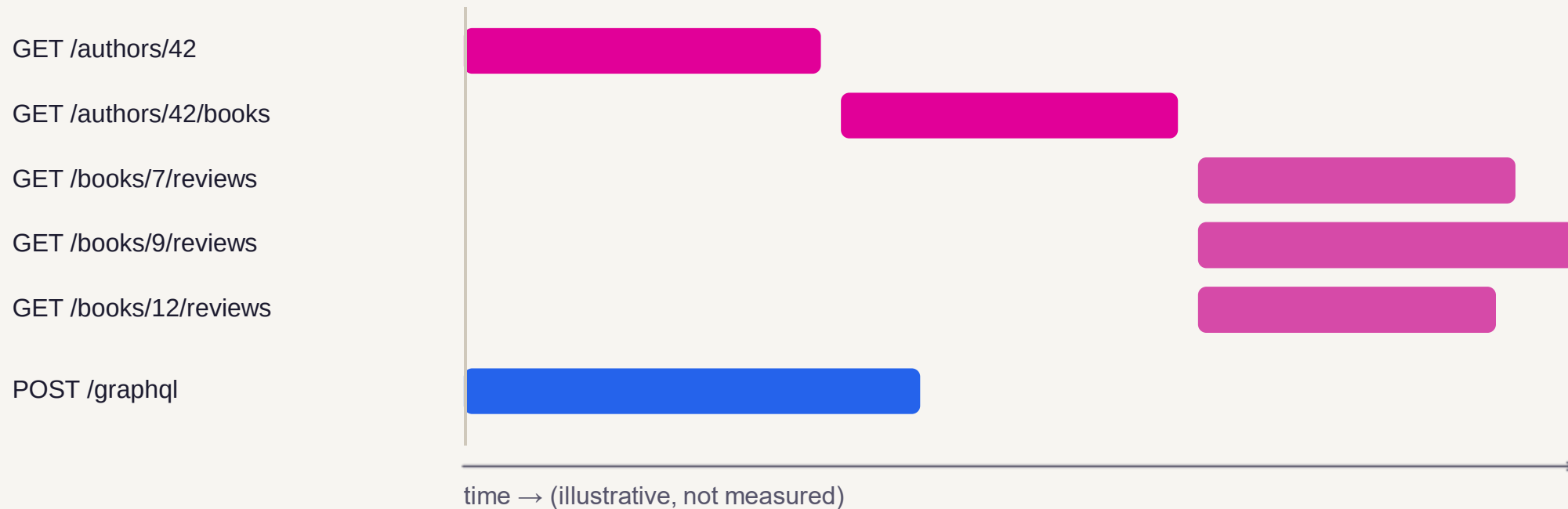
The screen uses **3 of 12** fields. The dimmed lines are downloaded, parsed and held in memory for nothing.

Why it happens: a resource endpoint serves every client at once — web, iOS, Android, partners — so it returns the union of everyone's needs.

Why it hurts: bytes on slow mobile networks, serialisation CPU on the server, and accidental exposure of fields no one intended to show.

UNDER-FETCHING

Under-fetching turns one screen into a request waterfall



Under-fetching forces dependent requests: you cannot ask for reviews until you know the book IDs. Latency adds up as a **waterfall**. A query language lets the client describe the whole tree so the server resolves it in one trip.

ENDPOINT SPRAWL

The usual workarounds each trade one problem for another



Custom endpoints

`/authors/42/profile-screen`
returns exactly what one screen needs — until the screen changes and the endpoint must change with it.



Query-string field filters

`?fields=name,avatar&include=books` — every API invents its own mini-language, with no shared tooling or validation.



Backend-for-frontend

A server per client (web BFF, mobile BFF) that aggregates calls. Effective, but multiplies services to build and operate.



Versioned endpoints

`/v1/`, `/v2/`... to change shapes safely. Clients migrate slowly; old versions linger for years.

THE PATTERN

Each workaround re-invents parts of a query language: client-chosen fields, nested includes, typed contracts. GraphQL standardises all of them in one specification.

DEFINITION

What an API query language is

An **API query language** is a formal language in which the **client** declares precisely which data it needs — and in what shape — and the **server** evaluates that declaration against a typed model of its data.



Declarative

The client says **what**, not **how**. It never specifies joins, endpoints or call order.



Shape-matching

The response mirrors the request's structure, so the client knows the result shape before it arrives.

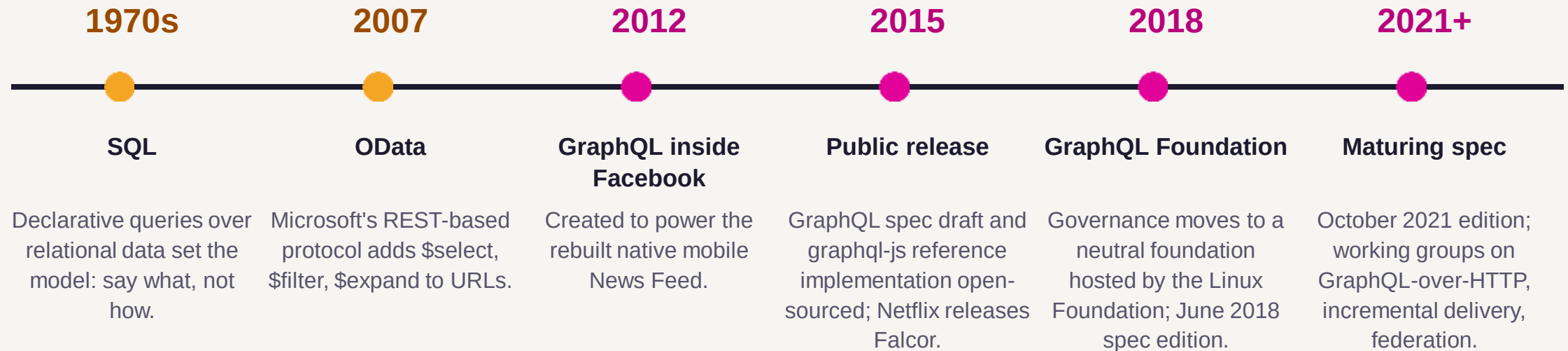


Typed & introspectable

A schema defines every type and field, enabling validation, autocomplete and generated code.

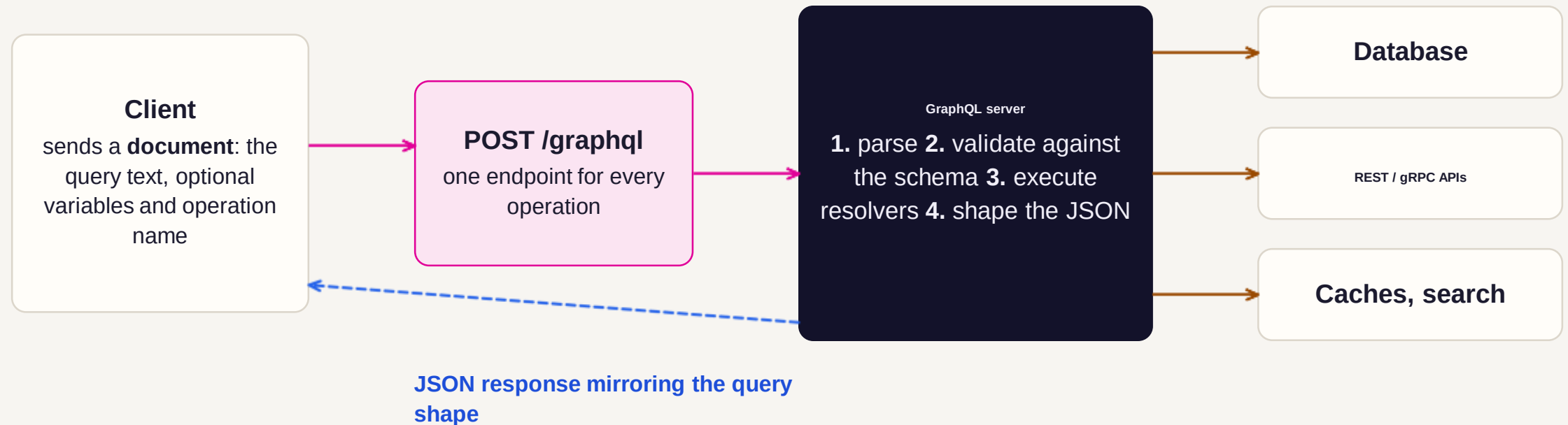
HISTORY

From SQL to GraphQL: fifty years of declarative data access



GRAPHQL AT A GLANCE

One endpoint, one typed schema, answers shaped by the client



GraphQL does **not** store data or replace your database. It is a query layer: a typed schema describes what can be asked, and resolver functions fetch each field from wherever the data actually lives.

GRAPHQL AT A GLANCE

The same screen, requested both ways

Resource-oriented (REST)

```
GET /authors/42
GET /authors/42/books
GET /books/7/reviews
GET /books/9/reviews
GET /books/12/reviews
```

5 requests · client joins data
every field of every resource

GraphQL

```
query AuthorScreen {
  author(id: "42") {
    name
    avatarUrl
    books(first: 3) {
      title
      reviewCount
    }
  }
}
```

One request, exactly the four facts the screen needs, and the response arrives nested in the same shape as the query.

THE LANDSCAPE

REST vs GraphQL vs gRPC: different tools, different strengths

Dimension	REST	GraphQL	gRPC
Contract	Optional (OpenAPI)	Required: typed schema (SDL)	Required: Protocol Buffers
Endpoints	Many — one per resource	Usually one	One per service method
Who shapes the response	Server	Client, within the schema	Server (message types)
Transport	HTTP	Usually HTTP; WebSocket/SSE for subscriptions	HTTP/2
HTTP caching	Native (GET + URLs)	Needs GET queries or persisted queries	Not HTTP-cache friendly
Real-time	Webhooks, SSE, polling	Subscriptions	Bidirectional streaming
Sweet spot	Public resource APIs, simple CRUD	Many clients, varied UIs, connected data	Internal service-to-service, low latency

JUDGEMENT

When GraphQL is probably not the best choice

- 01 Simple, stable CRUD** with one client: REST with good HTTP caching is less machinery.
- 02 File-heavy APIs** — binary upload and download are outside the core spec.
- 03 Very high-throughput internal RPC** where gRPC's binary encoding and streaming matter more than flexible shapes.
- 04 Public APIs that rely on CDN caching** of anonymous GET traffic, unless you adopt persisted queries.

THE TRADE

GraphQL moves complexity from the client to the server. You gain flexibility, and you must now handle query cost, caching and N+1 performance deliberately — Sections 11 and 12.

NOT EITHER/OR

Many organisations run both: GraphQL for product-facing graphs, REST or gRPC for internal and partner integrations.

VOCABULARY

Ten terms you will meet on every slide from here on

Term	Meaning
Schema	The complete, typed description of what a GraphQL API can do
Type	A named shape: object, scalar, enum, input, interface or union
Field	A named unit of data on a type; may take arguments
Operation	A query, mutation or subscription sent by a client
Document	The text a client sends; contains operations and fragments
Selection set	The { ... } block listing which fields to fetch
Resolver	The server function that produces one field's value
Variable	A typed, named input (\$id) passed separately from the document
Fragment	A reusable, named selection set
Directive	An @annotation that changes execution or carries metadata

CHECKPOINT 01

Test yourself before moving on

Q1. Why does a resource-oriented API tend to over-fetch?

Q2. What causes a request waterfall?

Q3. Name the three core properties of an API query language.

Q4. Give one situation where REST is the better fit.

Answers are in the speaker notes. Try answering aloud before you look.

SECTION 02 · FOUNDATION

Your first queries

The query language itself, one construct at a time: what you send, what comes back, and the rules that connect them.

IN THIS SECTION

- 01 Anatomy of a request
- 02 Anatomy of a response
- 03 Fields and selection sets
- 04 Shape mirroring
- 05 Arguments and literal values
- 06 Aliases
- 07 Operation types and names
- 08 Lexical rules
- 09 `__typename` and common errors

What actually travels over the wire

```
POST /graphql HTTP/1.1
Content-Type: application/json
Accept: application/graphql-response+json
```

```
{
  "query": "query Hi($n: String) { hello(name: $n) }",
  "variables": { "n": "Ada" },
  "operationName": "Hi"
}
```

1

1. One endpoint, usually POST

Every operation hits one URL; GET also works.

2

2. query (required)

The GraphQL document, as a string.

3

3. variables (optional)

JSON values for the \$variables.

4

4. operationName (optional)

Picks which operation in the document to run.

Every response is a JSON map with up to three keys

```
{
  "data": {
    "hello": "Hello, Ada!"
  },
  "errors": [
    {
      "message": "...",
      "locations": [{ "line": 1, "column": 3 }],
      "path": ["hello"]
    }
  ],
  "extensions": { "requestId": "a1b2" }
}
```

1

1. data

The result, in the same shape as the query.

2

2. errors

Only when something failed; may sit beside data.

3

3. locations and path

Where in the document and response it failed.

4

4. extensions

Free-form map: tracing, cost, request IDs.

HELLO, GRAPHQL

Your first query and the smallest valid response

```
query {  
  hello  
}
```



the server executes the
hello field

```
{  
  "data": {  
    "hello": "world"  
  }  
}
```

This is the smallest useful GraphQL program. The keyword **query** declares the operation type, the braces are a **selection set**, and **hello** is a **field** on the schema's root **Query** type. The response wraps the answer inside **data**, under the same key you asked for.

Shorthand: when an operation is an unnamed query with no variables, the keyword can be dropped entirely: **{ hello }**

FIELDS

Every field is either a leaf or a branch

```
query {  
  me {           # branch: object type  
    name         # leaf: String  
    email        # leaf: String  
    role         # leaf: enum  
    address {    # branch: object type  
      city       # leaf  
    }  
  }  
}
```

Leaf fields

Return a **scalar** (Int, Float, String, Boolean, ID, custom) or an **enum**. They must NOT have a selection set.

Branch fields

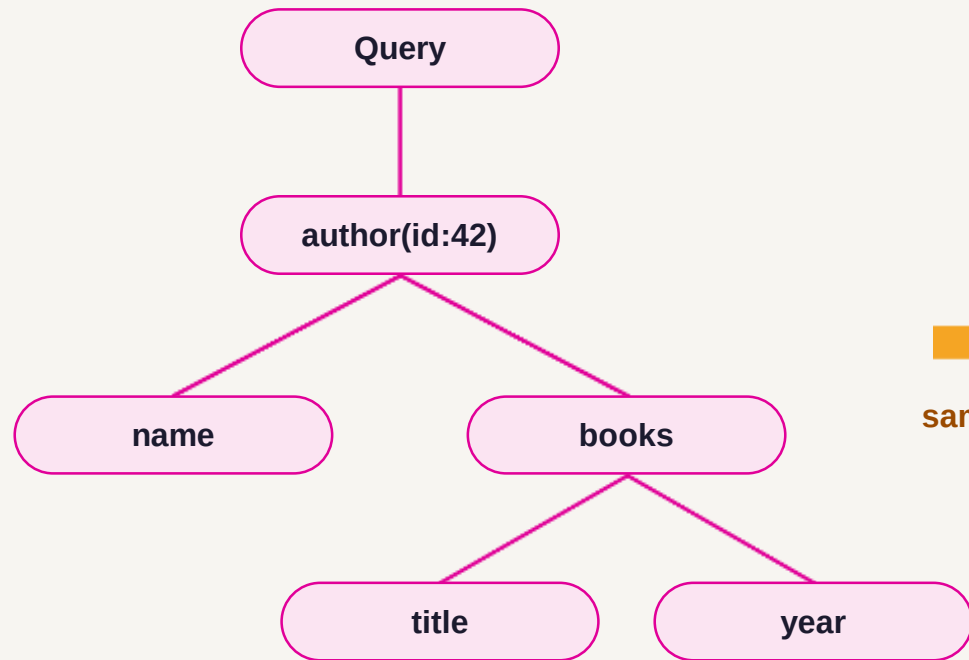
Return an **object**, **interface** or **union**. They **MUST** have a selection set; you choose which sub-fields you want.

There is no **SELECT ***. You always name each field you need — that is what makes queries self-documenting and cheap to analyse.

SELECTION SETS

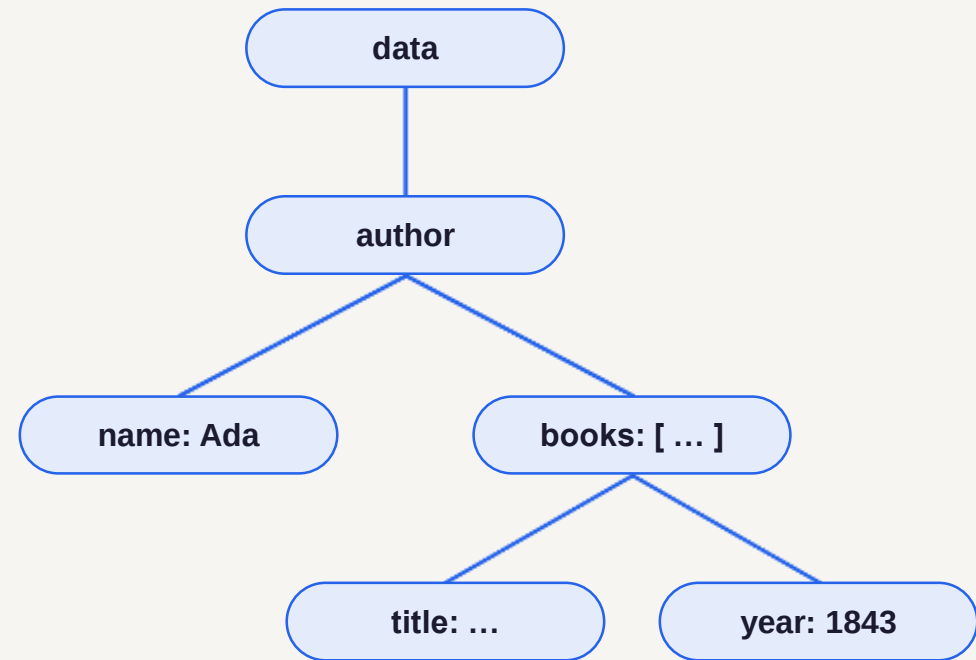
A query is a tree, and the response is the same tree filled in

Query tree — what you ask



same shape

Response tree — what you get



SHAPE MIRRORING

Every field you select becomes a key in the same place

```
query {  
  author(id: "42") {  
    name  
    books {  
      title  
      year  
    }  
  }  
}
```

```
{  
  "data": {  
    "author": {  
      "name": "Ada Byron",  
      "books": [  
        { "title": "Notes", "year": 1843 },  
        { "title": "Sketch", "year": 1842 }  
      ]  
    }  
  }  
}
```

Order is preserved: response keys appear in the same order as the fields in your selection set. **Lists become arrays:** **books** returns a list type, so every element is shaped by the same sub-selection.

ARGUMENTS

Arguments parameterise a field

```
query {  
  author(id: "42") {  
    name  
    avatarUrl(size: 64)  
    books(first: 3, orderBy: NEWEST) {  
      title  
    }  
  }  
}
```

- 01** Arguments are written **name: value** inside parentheses after the field name.
- 02** Order does not matter; names do. Each must be declared in the schema for that field.
- 03** **Any** field can take arguments — not just top-level ones. **avatarUrl(size: 64)** asks the server to transform data.
- 04** Required arguments are declared non-null (**id: ID!**); omitting one is a validation error.

ARGUMENTS

Seven kinds of literal value you can write inline

Kind	Example	Notes
Int	first: 10	32-bit signed integer; no quotes
Float	minRating: 4.5	Double-precision; 1e3 is also valid
String	title: "Dune"	Double quotes only; escapes like \n and \u00E9
Boolean	inStock: true	Lowercase true / false
Null	publisher: null	Explicitly null — different from omitting the argument
Enum	orderBy: NEWEST	Bare name, no quotes; must be a declared enum value
List	ids: ["1", "2"]	Commas optional; a single value is coerced to a one-item list
Input object	filter: { genre: SCIFI, year: 2020 }	Keys are the input type's fields

ALIASES

Aliases rename a field in the response

```
query {  
  newest: books(orderBy: NEWEST, first: 1) {  
    title  
  }  
  oldest: books(orderBy: OLDEST, first: 1) {  
    title  
  }  
  me {  
    displayName: name  
  }  
}
```

1

1. Syntax: alias: field

The alias becomes the response key instead of the field name.

2

2. Same field, different arguments

Without aliases these two would conflict: one key cannot hold two different results.

3

3. Renaming for the client

Match the names your UI code already expects.

ALIASES

What the aliased query returns

```
{
  "data": {
    "newest": [{ "title": "Tomorrow's Code" }],
    "oldest": [{ "title": "Notes on Engines" }],
    "me": { "displayName": "Ada" }
  }
}
```

THE RULE

Two fields may share a response key only if they are the **same field with identical arguments** — a validation rule called **fields in set can merge**.

Aliases are purely client-side naming: the server runs the same resolver either way.

OPERATIONS

Three operation types, one syntax

```
query AuthorScreen {  
  author(id: "42") { name }  
}  
  
mutation RateBook {  
  rateBook(id: "7", stars: 5) { id }  
}  
  
subscription OnReview {  
  reviewAdded(bookId: "7") { stars }  
}
```

1

1. query

Read-only fetch. Top-level fields may execute in parallel.

2

2. mutation

A write followed by a fetch. Top-level fields run one at a time, in order.

3

3. subscription

A long-lived stream of results as events happen.

OPERATIONS

Name every operation you ship

```
# Anonymous — fine for exploring
{ me { name } }

# Named — use in real code
query CurrentUser {
  me { name }
}
```

- 01 Debugging:** server logs and traces show **CurrentUser**, not an anonymous blob.
- 02 Metrics:** latency and error rates per operation name.
- 03 Multiple operations:** a document with more than one operation must name them all; the request's **operationName** picks one.
- 04 Persisted queries and code generation** key off the name.

Convention: PascalCase names that describe the use case — **ProductPage**, **AddToCart** — not the field fetched.

LEXICAL RULES

The small print that makes documents unambiguous

Rule	Detail
Names	/[_A-Za-z][_0-9A-Za-z]*/ — case-sensitive; names starting with __ are reserved for introspection
Whitespace & commas	Insignificant. { a, b }, { a b } and one-per-line are identical
Comments	# to end of line; ignored by the parser
Strings	"..." with escapes, or block strings """...""" that keep line breaks
Punctuators	! \$ & () ... : = @ [] { }
Source text	Unicode; documents are usually UTF-8
Keywords	query, type etc. are not reserved — a field can be called type

META-FIELDS

__typename tells you the concrete type of any object

```
query {  
  search(text: "ada") {  
    __typename  
    ... on Author { name }  
    ... on Book { title }  
  }  
}
```

```
{  
  "data": {  
    "search": [  
      { "__typename": "Author",  
        "name": "Ada Byron" },  
      { "__typename": "Book",  
        "title": "Notes" }  
    ]  
  }  
}
```

Available on every object, interface and union — even when the schema does not declare it. Client caches use it to identify records; you will use it heavily with unions in Section 07.

TROUBLESHOOTING

Four validation errors every beginner meets

You wrote...	The server says (paraphrased)	Fix
{ author(id: "1") }	Field of object type must have a selection of subfields	Add { name }
{ me { name { first } } }	Leaf field must not have a selection	Remove the braces
{ me { nmae } }	Cannot query field on type — often with a suggestion	Check spelling or schema
{ author { name } }	Required argument id was not provided	Pass id:

Validation errors are returned before any resolver runs. The response has an **errors** array and no **data** key at all.

CHECKPOINT 02

Test yourself before moving on

Q1. What is the difference between a leaf and a branch field?

Q2. How do you fetch the same field twice with different arguments?

Q3. What three keys can a JSON request body carry?

Q4. Is { a, b } different from { a b }?

Answers are in the speaker notes.

SECTION 03 · FOUNDATION

Variables & directives

Make documents reusable and safe with typed variables, then control execution at runtime with directives.

IN THIS SECTION

- 01 Why variables exist
- 02 Declaring variables
- 03 Sending variables
- 04 Allowed variable types
- 05 Defaults, null and omission
- 06 Variable usage rules
- 07 Directives: syntax and locations
- 08 @include and @skip
- 09 Built-in directive reference

VARIABLES

Never build a GraphQL document with string concatenation

Don't — interpolate user input

```
const q = `{  
  books(title: "${userInput}") {  
    title  
  }  
}`  
// input: x") { id } secret { key  
// → a different, injected query
```

Do — declare a variable

```
const q = `query Find($t: String!) {  
  books(title: $t) { title }  
}`  
  
fetch(url, { body: JSON.stringify({  
  query: q, variables: { t: userInput }  
}}))
```



Safe

Values are never parsed as GraphQL syntax.



Cacheable

The document text is constant, so servers can cache its parsed and validated form.



Typed

Each variable is validated and coerced against its declared type.

VARIABLES

Declaring variables in the operation definition

```
query BooksByAuthor(  
  $authorId: ID!,  
  $first: Int = 10,  
  $genre: Genre  
) {  
  author(id: $authorId) {  
    books(first: $first, genre: $genre) {  
      title  
    }  
  }  
}
```

- 1. \$name: Type**
Declared in parentheses after the operation name. The \$ prefix is required.
- 2. Default value**
Used when the client omits the variable.
- 3. Nullable = optional**
No ! means the variable may be omitted or null.
- 4. Use anywhere a value fits**
Pass variables as argument values at any depth in the operation.

VARIABLES

The document and the variables travel separately

```
document (constant)

query BooksByAuthor(
  $authorId: ID!,
  $first: Int = 10
) { ... }
```

```
variables (per request)

{
  "authorId": "42",
  "first": 3
}
```

Coerce & validate
each value against its
declared type

Execute
with the coerced values
bound to \$authorId, \$first

If a required variable is missing, or a value cannot be coerced ("abc" for Int), the request fails with a **request error** before execution starts: no **data** key is returned.

VARIABLES

Variables must have input types

Allowed	Example
Scalars	\$id: ID!, \$price: Float
Enums	\$sort: SortOrder = ASC
Input objects	\$filter: BookFilter
Lists of the above	\$ids: [ID!]!

NOT ALLOWED

Object, interface and union types are **output** types. They can never be the type of a variable or argument.

```
query Search(  
  $ids: [ID!]!,  
  $filter: BookFilter,  
  $sort: SortOrder = ASC  
) {  
  books(ids: $ids,  
    filter: $filter,  
    sort: $sort) { title }  
}
```

VARIABLES

Omitted, null and defaults are three different things

Declaration	Client sends	Value seen by the server
\$first: Int = 10	(omitted)	10 — the default applies
\$first: Int = 10	"first": null	null — explicit null overrides the default
\$first: Int	(omitted)	No value; the argument is treated as not provided
\$first: Int!	(omitted)	Request error: required variable missing
\$first: Int!	"first": null	Request error: null not allowed
\$ids: [ID!]	"ids": "7"	["7"] — a single value coerces to a list

Why it matters: for an update mutation, **omitted** can mean “leave unchanged” while **null** means “clear this field”. Design and document the difference deliberately.

VARIABLES

A variable must fit every position it is used in

Variable	Argument	Valid?	Why
\$id: ID!	author(id: ID!)	Yes	Exact match
\$id: ID!	search(id: ID)	Yes	Non-null always fits a nullable position
\$id: ID	author(id: ID!)	No	Could be null in a non-null position
\$id: ID = "1"	author(id: ID!)	Yes	Allowed: the variable has a non-null default
\$n: Int	author(id: ID!)	No	Int and ID are different types
\$tag: String!	books(tags: [String!])	Yes	A single value is allowed in a list position

Every declared variable must also be **used** somewhere in the operation, and every variable used must be **declared**. Both are validation rules.

VARIABLES

Input objects bundle related values into one variable

schema (SDL)

```
input BookFilter {  
  genre: Genre  
  publishedAfter: Int  
  author: String  
}
```

operation

```
query Filtered($f: BookFilter!) {  
  books(filter: $f) { title year }  
}
```

variables

```
{  
  "f": {  
    "genre": "SCIFI",  
    "publishedAfter": 2015  
  }  
}
```

Enum values in JSON variables are **strings** ("SCIFI"); in the document they are bare names (SCIFI).

Unknown keys in an input object are rejected, which catches typos early.

DIRECTIVES

Directives annotate a document or schema with extra behaviour

syntax

field @name

field @name(arg: value)

field @a @b(x: 1)

A directive is an @name with optional arguments, attached to a location. It tells the server — or a tool — to treat that location differently.



Executable locations

In operations: QUERY, MUTATION, SUBSCRIPTION, FIELD, FRAGMENT_DEFINITION, FRAGMENT_SPREAD, INLINE_FRAGMENT, VARIABLE_DEFINITION.



Type-system locations

In the schema: SCHEMA, SCALAR, OBJECT, FIELD_DEFINITION, ARGUMENT_DEFINITION, INTERFACE, UNION, ENUM, ENUM_VALUE, INPUT_OBJECT, INPUT_FIELD_DEFINITION.

DIRECTIVES

@include and @skip toggle fields at request time

```
query Book(  
  $id: ID!,  
  $withReviews: Boolean!,  
  $compact: Boolean!  
) {  
  book(id: $id) {  
    title  
    reviews @include(if: $withReviews) {  
      stars  
    }  
    description @skip(if: $compact)  
  }  
}
```

1

1. Boolean variables drive it

One document serves many views.

2

2. @include(if: ...)

Field is fetched only when the condition is true.

3

3. @skip(if: ...)

Field is omitted when the condition is true.

4

4. Absent, not null

A skipped field has no key in the response at all.

DIRECTIVES

When @include and @skip meet on one field

@skip(if:)	@include(if:)	Field fetched?
false	true	Yes
false	false	No
true	true	No
true	false	No

DIRECTIVES

Built-in directives you should know

Directive	Where	Purpose	Status
@include(if: Boolean!)	Field, fragment	Fetch only when true	Spec
@skip(if: Boolean!)	Field, fragment	Omit when true	Spec
@deprecated(reason: String)	Schema: fields, enum values, args, input fields	Mark as retiring; tools warn users	Spec
@specifiedBy(url: String!)	Schema: custom scalars	Link to the scalar's format spec	Spec (2021)
@oneOf	Schema: input objects	Exactly one field must be set	Newer addition — check server support
@defer / @stream	Fragments / list fields	Incremental delivery of slow parts	Proposal; supported by some servers

Custom directives — like **@auth** or **@cacheControl** — are defined by servers and tools. We build one in Section 07.

CHECKPOINT 03

Test yourself before moving on

Q1. Why must user input go in variables rather than into the document string?

Q2. Can a variable have type Book (an object type)?

Q3. \$first: Int = 10 and the client sends null. What value is used?

Q4. A field has @skip(if: false) @include(if: false). Is it fetched?

Answers are in the speaker notes.

SECTION 04 · CORE

Fragments

Name a selection set once, reuse it everywhere, and select fields that exist only on certain types.

IN THIS SECTION

- 01 The repetition problem
- 02 Named fragments
- 03 How spreads expand
- 04 Type conditions
- 05 Inline fragments
- 06 Interfaces and unions
- 07 Composing fragments
- 08 Colocation with UI components
- 09 Validation rules

FRAGMENTS

Without fragments, the same fields are repeated everywhere

```
query Home {  
  featured { title coverUrl  
    author { name } }  
  bestsellers { title coverUrl  
    author { name } }  
  newReleases { title coverUrl  
    author { name } }  
}
```

Three lists all render the same **book card**. The card's data needs are copied three times.

MAINTENANCE COST

Add a **price** to the card and you must edit three places — and remember every other screen that shows a card.

A **fragment** gives that selection a name so it lives in exactly one place.

FRAGMENTS

Named fragments: define once, spread anywhere

```
fragment BookCard on Book {  
  title  
  coverUrl  
  author { name }  
}  
  
query Home {  
  featured { ...BookCard }  
  bestsellers { ...BookCard }  
  newReleases { ...BookCard }  
}
```

1

1. fragment Name on Type

Defines a named selection set for one type.

2

2. The body

Any fields, arguments, aliases or nested selections.

3

3. ...Name spreads it

Three dots plus the name: the fields are inserted here.

FRAGMENTS

A spread is replaced by its fields before execution

```
fragment BookCard on Book {  
  title  
  coverUrl  
}
```

```
featured {  
  ...BookCard  
}
```



server inlines at
validation

```
featured {  
  title  
  coverUrl  
}
```

Effective selection. The response contains **title** and **coverUrl** directly — there is no trace of the fragment in the data.

TYPE CONDITIONS

Every fragment declares the type it applies to

```
fragment Named on Node {  
  id  
}  
fragment AuthorBits on Author {  
  name  
  bornIn  
}
```

- 01** **on Book** is the **type condition**. The fragment's fields are only selected when the runtime object is that type.
- 02** The condition can be an **object**, an **interface** or a **union**.
- 03** Spreading a fragment where the types can never overlap — **...AuthorBits** inside a **Book** — is a validation error.
- 04** This is what makes fragments safe: the server proves every field exists wherever the fragment is used.

INLINE FRAGMENTS

Inline fragments select type-specific fields in place

```
query Search($q: String!) {  
  search(text: $q) {  
    __typename  
    ... on Book { title year }  
    ... on Author { name }  
  }  
  book(id: "7") {  
    title  
    ... @include(if: false) {  
      isbn  
      pageCount  
    }  
  }  
}
```

1

1. ... on Type { }

An anonymous fragment written in place.

2

2. One branch per possible type

Fields apply only to objects of that type.

3

3. No type condition

Group fields to apply one directive to all of them.

POLYMORPHISM

Querying an interface: shared fields plus type-specific ones

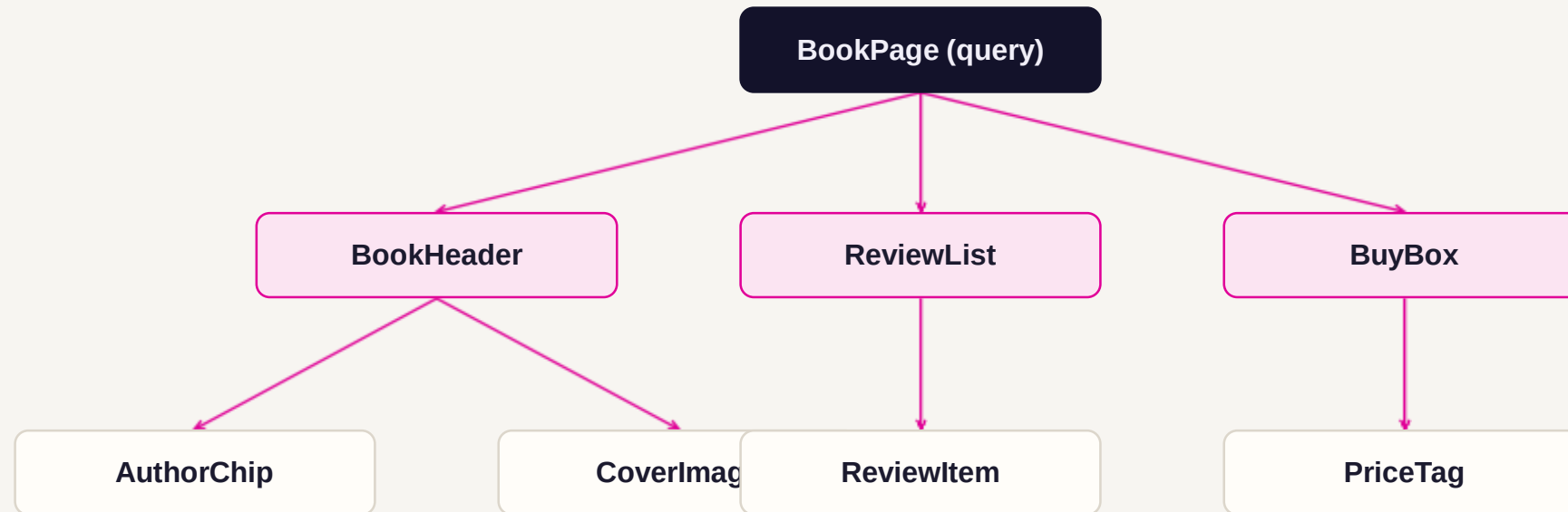
```
query Feed {  
  feed {           # [FeedItem!]!  
    id             # on interface  
    createdAt      # on interface  
    ... on ReviewItem {  
      stars  
      book { title }  
    }  
    ... on NewBookItem {  
      book { title coverUrl }  
    }  
  }  
}
```

```
{"data": {"feed": [  
  { "id": "r1",  
    "createdAt": "2026-09-01",  
    "stars": 5,  
    "book": { "title": "Notes" } },  
  { "id": "n9",  
    "createdAt": "2026-09-02",  
    "book": { "title": "Sketch",  
              "coverUrl": "..." } }  
]}}
```

Fields declared on the interface can be selected directly. Fields that exist only on one implementation need a fragment with that type condition.

COMPOSITION

Fragments compose into a tree that mirrors your UI



Fragments can spread other fragments. The page query spreads **...BookHeader**, which spreads **...AuthorChip** and **...CoverImage**. The final document is one tree, assembled from small, named pieces.

COLOCATION

Colocate each component with the fragment it needs

```
// ReviewItem.tsx
export const ReviewItemFragment = gql`
  fragment ReviewItem on Review {
    stars
    body
    author { name }
  }
`

function ReviewItem({ review }) {
  return <p>{review.stars}★ {review.body}</p>
}
```

- 01** The component that renders data **declares** that data, next to its code.
- 02** Parents spread children's fragments; they never need to know the children's fields.
- 03** Delete a component and its fragment goes with it — no orphaned fields in queries.
- 04** Relay enforces this with **data masking**: a parent cannot read fields requested by a child's fragment.

VALIDATION

Five rules every fragment must satisfy

Rule	Invalid example	Why
Names are unique	two fragment Card ... in one document	Spreads must be unambiguous
Every fragment is used	fragment Unused on Book {...} never spread	Dead code is rejected, not ignored
No cycles	A spreads B, B spreads A	Expansion would never terminate
Type must exist and be composite	fragment X on String	Only objects, interfaces, unions have fields
Spread must be possible	...AuthorBits inside book { }	Book and Author can never overlap

CHECKPOINT 04

Test yourself before moving on

Q1. What does `...BookCard` do at execution time?

Q2. When do you need an inline fragment on an interface field?

Q3. Why write `... @include(if: $x) { a b }` with no type condition?

Q4. What is fragment colocation?

Answers are in the speaker notes.

SECTION 05 · CORE

Mutations

Change data on the server and fetch the result in the same round trip — with predictable ordering and well-designed errors.

IN THIS SECTION

- 01 What a mutation is
- 02 Input objects
- 03 Serial execution
- 04 Several mutations at once
- 05 Returning what changed
- 06 The payload pattern
- 07 Two kinds of errors
- 08 Naming and design conventions

MUTATIONS

A mutation is a write followed by a query

```
mutation AddReview($input: AddReviewInput!) {  
  addReview(input: $input) {  
    review {  
      id  
      stars  
    }  
    book {  
      id  
      reviewCount  
    }  
  }  
}
```

1

1. mutation keyword

Declares an operation with side effects.

2

2. Root field = the action

Defined on the schema's Mutation type.

3

3. Then a normal selection

Choose what to read back after the write.

4

4. Read related data too

Fetch the updated count in the same trip.

MUTATIONS

Input objects keep mutation arguments tidy and evolvable

```
schema
input AddReviewInput {
  bookId: ID!
  stars: Int!
  body: String
}

type Mutation {
  addReview(input: AddReviewInput!):
    AddReviewPayload!
}
```

```
variables
{
  "input": {
    "bookId": "7",
    "stars": 5,
    "body": "Visionary."
  }
}
```

CONVENTION

One **input** argument per mutation: adding an optional field later is non-breaking, and clients pass one variable.

EXECUTION ORDER

Queries may run in parallel; mutations run one by one

Query — top-level fields may run in parallel



Mutation — top-level fields run strictly in order



The spec requires a mutation's **top-level** fields to execute **serially**: the second starts only after the first — and its entire sub-selection — completes. Sub-selections inside each field may still resolve in parallel. This lets you chain dependent writes in one request and know the order they happened in.

Serial ≠ transactional. If `applyCoupon` fails, `addToCart` has already happened. Transactions are your resolver's job.

MUTATIONS

Several mutations in one request, with aliases

```
mutation Checkout($c: ID!) {  
  first: addToCart(itemId: "7") {  
    cart { total }  
  }  
  second: addToCart(itemId: "9") {  
    cart { total }  
  }  
  order: checkout(cartId: $c) {  
    order { id status }  
  }  
}
```

```
{"data": {  
  "first": {"cart": {"total": 12.5}},  
  "second": {"cart": {"total": 30.0}},  
  "order": {"order": {  
    "id": "o-88",  
    "status": "PLACED" }}  
}}
```

Aliases are required here: **addToCart** appears twice with different arguments. The totals prove the order — 12.5 before 30.0.

MUTATIONS

Return the changed objects — with their IDs

```
mutation Rename {  
  renameBook(id: "7", title: "Notes, 2nd ed.") {  
    book {  
      id      # identity for the cache  
      title   # the new value  
      updatedAt  
    }  
  }  
}
```

- 01 Clients with a **normalised cache** (Apollo, Relay, urql Graphcache) match objects by **__typename** + **id** and update every screen that shows this book automatically.
- 02 Returning the object avoids a second, follow-up query.
- 03 Returning only **Boolean** (“success: true”) throws that away — avoid it.

The payload pattern: results and expected errors, side by side

```
type AddReviewPayload {  
  review: Review  
  book: Book  
  userErrors: [UserError!]!  
}  
  
type UserError {  
  field: [String!]  
  message: String!  
  code: ReviewErrorCode!  
}
```

1

1. Result objects

Nullable: absent when the mutation fails.

2

2. Expected failures as data

Always a list; empty on success.

3

3. Which input was wrong

A path such as ["input", "stars"] for form UIs.

4

4. Machine-readable code

An enum clients can switch on.

ERROR DESIGN

Two different kinds of failure, two different channels



Top-level errors

Unexpected or technical failures: server exceptions, timeouts, authentication missing, invalid documents. Clients generally cannot recover in-app; they log and show a generic message.



userErrors in the payload

Expected domain outcomes: “stars must be 1–5”, “book already reviewed”, “coupon expired”. Typed, documented in the schema, and rendered next to the form field.

```
{
  "data": {
    "addReview": {
      "review": null,
      "userErrors": [
        {
          "field": ["input", "stars"],
          "message": "Stars must be between 1 and 5",
          "code": "OUT_OF_RANGE"
        }
      ]
    }
  }
}
```

An alternative is a **result union** — `union AddReviewResult = AddReviewSuccess | InvalidStars | AlreadyReviewed` — covered in Section 13.

DESIGN

Mutation naming and design conventions



verbNoun names

addReview, **cancelOrder**, **publishBook**. Group by noun in tools by using consistent verbs.



Specific over generic

publishBook beats **updateBook(status: PUBLISHED)**: intent is explicit, permissions and validation are simpler.



One input, one payload

input: XInput! in, **XPayload!** out. Both can grow without breaking clients.



Idempotency

For retries on flaky networks, accept a client-generated key (for example **clientMutationId** or an idempotency key) and de-duplicate.

CHECKPOINT 05

Test yourself before moving on

Q1. How do mutation top-level fields execute?

Q2. Does serial execution make a mutation transactional?

Q3. Why return the updated object (with id) from a mutation?

Q4. Where should “coupon expired” go: errors or userErrors?

Answers are in the speaker notes.

SECTION 06 · CORE

Subscriptions

Instead of asking repeatedly, the client subscribes once and the server pushes a result every time a relevant event occurs.

IN THIS SECTION

- 01 Subscription syntax
- 02 The single-root-field rule
- 03 Lifecycle of a subscription
- 04 Transport protocols
- 05 Server architecture with pub/sub
- 06 Writing a subscription resolver
- 07 Subscriptions vs polling vs live queries

SUBSCRIPTIONS

Subscribe once, receive a result per event

```
subscription OnReview($bookId: ID!) {  
  reviewAdded(bookId: $bookId) {  
    id  
    stars  
    author { name }  
    book { reviewCount }  
  }  
}
```

1
2
3

1. subscription keyword

Opens a long-lived operation.

2. Exactly one root field

The event source you listen to.

3. A normal selection set

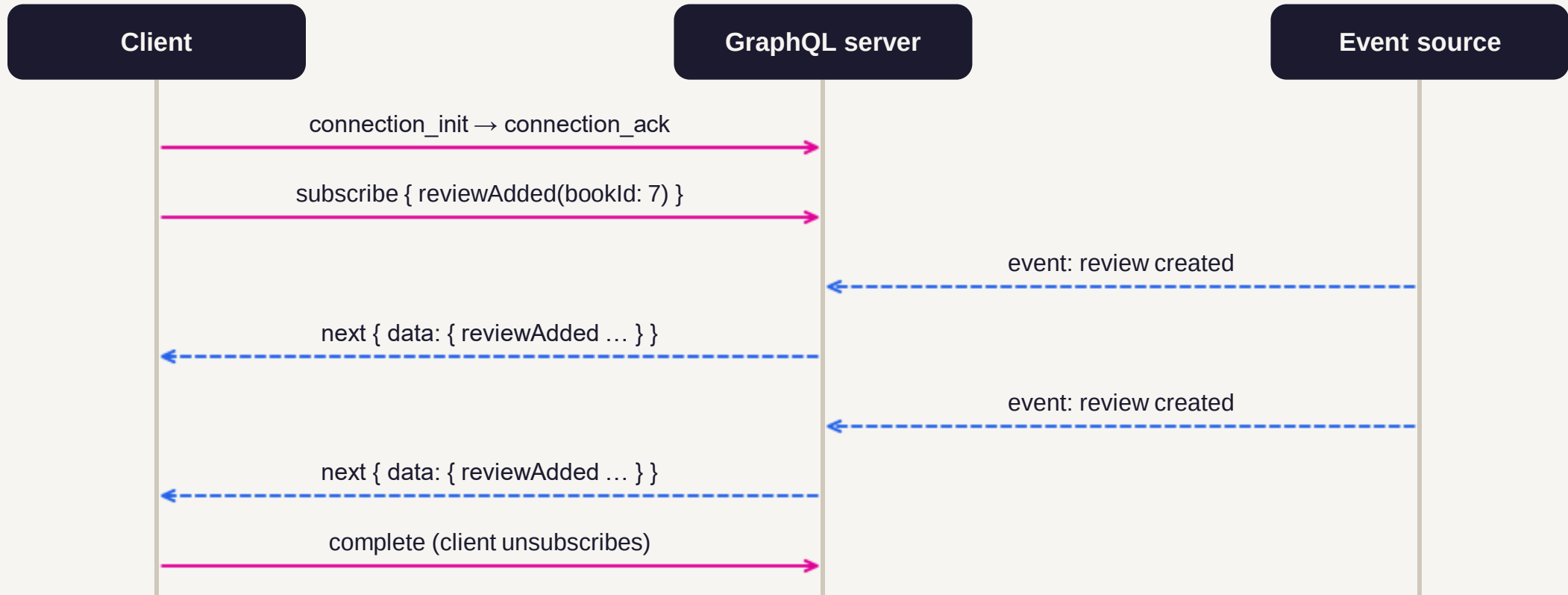
Shapes each pushed result.

Each time a review is added for that book, the server runs the selection set against the event and pushes a complete result shaped like any query response: `{ "data": { "reviewAdded": { ... } } }`.

Rule: a subscription operation must select exactly one top-level field (introspection fields such as `__typename` are not allowed at the root either).

LIFECYCLE

The life of one subscription over a WebSocket



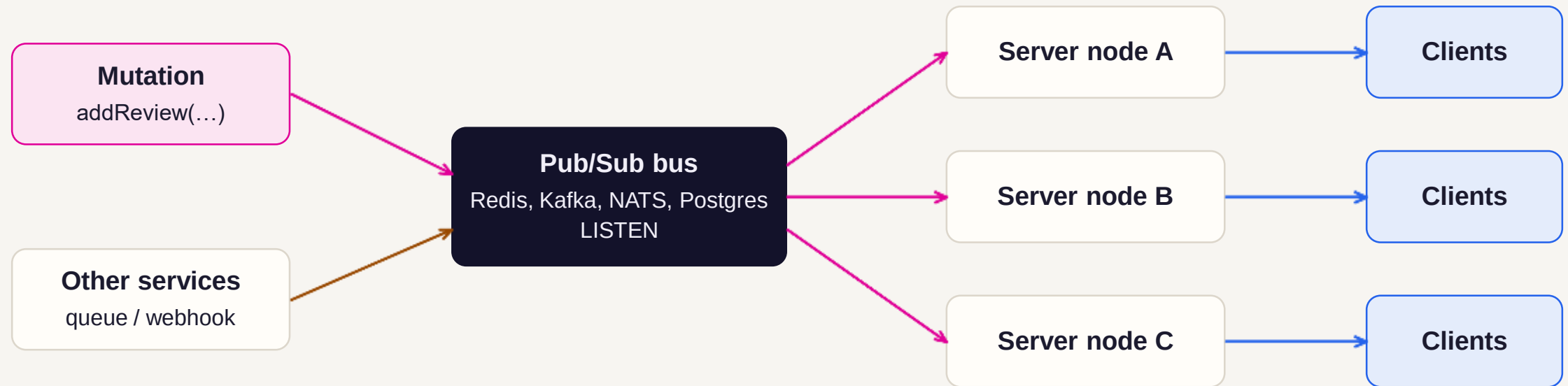
TRANSPORTS

The spec defines semantics; transports are separate standards

Transport	How it works	Good for
WebSocket — graphql-ws protocol	Bidirectional socket; messages such as <code>connection_init</code> , <code>subscribe</code> , <code>next</code> , <code>error</code> , <code>complete</code>	Browsers and apps needing many concurrent subscriptions
Server-Sent Events (SSE)	One-way HTTP stream from server to client; plain HTTP infrastructure	Simpler deployments, HTTP/2 multiplexing, proxies that dislike WebSockets
Multipart HTTP responses	A streamed multipart/mixed response carrying successive payloads	Clients already using incremental delivery (@defer)

Beware older libraries: `subscriptions-transport-ws` is a legacy protocol, incompatible with `graphql-ws`. Client and server must speak the same one.

Scaling subscriptions requires a shared event bus



With more than one server instance, an in-memory event emitter is not enough: an event raised on node A must reach subscribers connected to nodes B and C. A shared **pub/sub bus** fans each event out to every node, which filters it and pushes it to its own subscribers.

IMPLEMENTATION

A subscription resolver has two parts: subscribe and resolve

```
const resolvers = {
  Subscription: {
    reviewAdded: {
      // 1. return an async iterator of events
      subscribe: (_, { bookId }, { pubsub }) =>
        pubsub.subscribe(`REVIEW:${bookId}`),
      // 2. map each event to the field value
      resolve: (event) => event.review,
    },
  },
  Mutation: {
    addReview: async (_, { input }, ctx) => {
      const review = await ctx.db.reviews.insert(input)
      ctx.pubsub.publish(`REVIEW:${input.bookId}`, { review })
      return { review, userErrors: [] }
    },
  },
}
```

- 01** **subscribe** returns an **async iterable**; each value becomes one pushed result.
- 02** **resolve** maps the raw event to the field's value; the rest of the selection resolves normally.
- 03** Publish **after** the write commits, so subscribers never see phantom data.

CHOOSING

Subscriptions, polling or live queries?

Approach	How	Use when	Costs
Polling	Re-run a query every N seconds	Data changes slowly; freshness of seconds is fine	Wasted requests; simple to operate
Subscriptions	Subscribe to specific events	Small, incremental updates: chat, notifications, prices	Stateful connections; pub/sub infrastructure
Live queries (non-standard)	Server re-sends a query's result when it changes	You want “this query, always fresh” semantics	Server must track dependencies; limited support

PRAGMATIC DEFAULT

Start with polling. Reach for subscriptions when the latency or request volume of polling becomes a measured problem.

CHECKPOINT 06

Test yourself before moving on

Q1. How many root fields may a subscription operation select?

Q2. Why is an in-memory event emitter insufficient with three server instances?

Q3. What are the two parts of a subscription resolver?

Q4. Name two transports for subscriptions.

Answers are in the speaker notes.

SECTION 07 · CORE

The type system & SDL

The schema is the heart of GraphQL. Learn to read and write every construct of the Schema Definition Language, and the rules behind them.

IN THIS SECTION

- 01 The schema as contract
- 02 Root operation types
- 03 Scalars, built-in and custom
- 04 Objects, fields, arguments
- 05 Nullability and null propagation
- 06 Lists
- 07 Enums and input types
- 08 Interfaces and unions
- 09 Descriptions, directives, extensions

THE SCHEMA

The schema is the single source of truth for the API

Every field a client can ask for, every argument it can pass, and every type it can receive is declared in the schema. **If it is not in the schema, it does not exist.**



Validation

Every request is checked against it before any code runs.



Documentation

Descriptions and types are browsable in every IDE.



Code generation

Typed clients and server stubs are generated from it.



Collaboration

Front-end and back-end agree on it first, then build in parallel.

Schema Definition Language: reading a first schema

```
"""A book in the catalogue."""  
type Book {  
  id: ID!  
  title: String!  
  year: Int  
  author: Author!  
  reviews(first: Int = 10): [Review!]!  
}  
  
type Query {  
  book(id: ID!): Book  
}
```

1

1. Description

Documentation that tools display.

2

2. type Name { field: Type }

Named fields, each with a return type.

3

3. Fields can take arguments

With types and optional defaults.

4

4. Query: the entry point

Its fields are the roots of every query.

ROOT TYPES

Three root operation types are the doors into your graph

```
schema {  
  query: Query  
  mutation: Mutation  
  subscription: Subscription  
}  
  
type Query    { books: [Book!]! }  
type Mutation { addBook(input: AddBookInput!): Book }  
type Subscription { bookAdded: Book! }
```

- 01 **Query** is mandatory; Mutation and Subscription are optional.
- 02 If the root types are named exactly Query, Mutation and Subscription, the **schema { }** block can be omitted.
- 03 Root types are ordinary object types — only their role is special.

TYPE SYSTEM MAP

Six named types, two directions, two wrappers

OUTPUT — what fields can return

Object
type Book { ... }

Interface
interface Node { ... }

Union
union Result = A | B

Scalar & Enum
String, Int, DateTime... enum
Genre { ... } — usable in BOTH
directions

INPUT — what arguments accept

Input object
input BookFilter { ... }

Wrapping types
[T] lists and T! non-null wrap any of these

There are six kinds of **named type**. Objects, interfaces and unions can only be returned; input objects can only be passed in; scalars and enums are **leaf** types usable on both sides. Lists and non-null are **wrapping** types, not named types.

SCALARS

Five built-in scalars

Scalar	Represents	Serialised as	Watch out for
Int	Signed 32-bit integer	JSON number	Values beyond $\pm 2,147,483,647$ are errors — use a custom scalar or String for large IDs, money in minor units
Float	Double-precision floating point	JSON number	Not for money: rounding errors
String	UTF-8 character sequence	JSON string	Unbounded length — validate it
Boolean	true or false	JSON boolean	
ID	Unique identifier, opaque to clients	JSON string	Accepts "7" or 7 as input; always output as a string

SCALARS

Custom scalars add domain-specific leaf types

```
scalar DateTime
  @specifiedBy(url:
    "https://www.rfc-editor.org/rfc/rfc3339")

type Review {
  createdAt: DateTime!
}
```

Common custom scalars: **DateTime**, **Date**, **URL**, **Email**, **JSON**, **BigInt**, **Decimal**, **UUID**. Libraries such as `graphql-scalars` ship ready-made ones for JavaScript.

serialize

internal value → JSON output (Date → "2026-09-28T10:00:00Z")

parseValue

JSON variable → internal value

parseLiteral

literal in the document (AST) → internal value

A custom scalar is **declared** in SDL and **implemented** in code with three functions. **@specifiedBy** links to the format, so clients know exactly what to send.

OBJECTS

Object types: fields, arguments and default values

```
type Author {  
  id: ID!  
  name: String!  
  books(  
    first: Int = 10  
    after: String  
    genre: Genre  
  ): [Book!]!  
  avatarUrl(size: Int = 128): String  
}
```

1

1. Object type

A named set of fields.

2

2. Arguments list

One per line; defaults are optional.

3

3. Return type

After the closing parenthesis.

4

4. Arguments on leaves

Even scalar fields can be parameterised.

NULLABILITY

In GraphQL, every type is nullable unless you say otherwise

```
type Book {  
  title: String! # never null  
  subtitle: String # may be null  
  author: Author! # never null  
}
```

THE MEANING OF !

! is a promise **from the server**: “this field will never be null.”
Breaking the promise triggers null propagation.

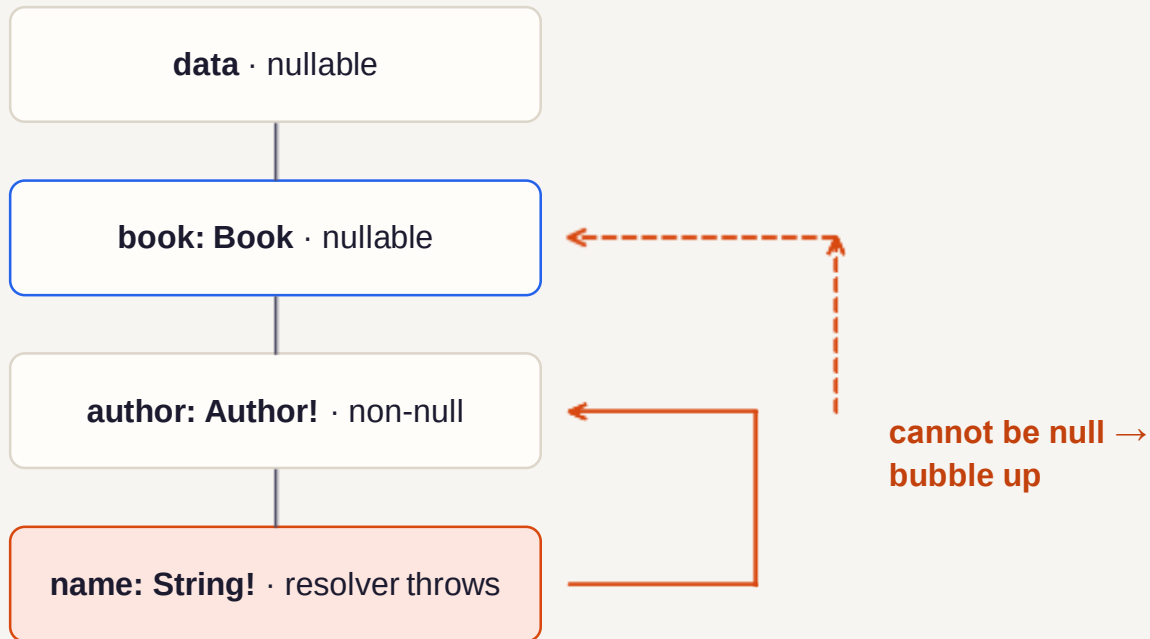
Why nullable by default? Each field may be resolved by a different system. If a reviews service is down, a nullable **reviews** field can be null while the rest of the page still renders.

Why add !? Clients no longer need null checks, and generated types become simpler.

Guideline: make a field non-null only when you are sure it can always be produced — removing a ! later is a **breaking change** for clients.

NULL PROPAGATION

A null in a non-null field bubbles up to the nearest nullable parent



```
{  
  "data": {  
    "book": null  
  },  
  "errors": [{  
    "message": "...",  
    "path": ["book",  
             "author", "name"]  
  }]  
}
```

A null in a non-null position **bubbles up** to the nearest nullable ancestor, which becomes null. Here **book** absorbs it. If no ancestor is nullable, all of **data** becomes null.

LISTS

Lists and non-null combine into four distinct contracts

Type	The list itself	Its items	Valid values
[Book]	may be null	may be null	null · [] · [b1, null]
[Book!]	may be null	never null	null · [] · [b1, b2]
[Book]!	never null	may be null	[] · [b1, null]
[Book!]!	never null	never null	[] · [b1, b2]

RULE OF THUMB

For collections, **[Book!]!** is the usual choice: the client always gets a list, possibly empty, with no null holes. Lists can nest, as in **[[Int!]!]!** for a matrix.

ENUMS

Enums restrict a value to a fixed, documented set

```
enum Genre {  
  FICTION  
  SCIFI  
  HISTORY  
  POETRY @deprecated(reason: "Use VERSE")  
  VERSE  
}  
  
type Book { genre: Genre! }  
type Query { books(genre: Genre): [Book!]! }
```

- 01 Values are conventionally SCREAMING_SNAKE_CASE.
- 02 In documents: bare **SCIFI**. In JSON variables and responses: the string "**SCIFI**".
- 03 Servers can map each value to an internal representation (a database code, an integer).
- 04 Adding a value can break clients that **switch** exhaustively — treat additions with care.

INPUT TYPES

Why inputs are a separate kind of type

	type (output object)	input (input object)
Used for	Field return values	Arguments and variables
Fields may have arguments	Yes	No
Field types allowed	Any output type	Scalars, enums, input objects, lists
Interfaces / unions	Can implement / be members	Not allowed
Default values	On arguments	On input fields
Cycles	Allowed (Author → Book → Author)	Only through a nullable or list field

Output objects may contain resolved, computed, recursive data; inputs must be finite, fully known values sent by a client. Hence the separation: typically **Book** for reading and **BookInput** for writing.

INTERFACES

Interfaces define fields that several types share

```
interface Node {  
  id: ID!  
}
```

```
interface Publication implements Node {  
  id: ID!  
  title: String!  
}
```

```
type Book implements Publication & Node {  
  id: ID!  
  title: String!  
  isbn: String  
}
```

1

1. interface

An abstract set of fields.

2

2. Interfaces can implement interfaces

Since the 2021 spec edition.

3

3. implements A & B

A type may implement several; listed with &.

4

4. Plus its own fields

Implementations add any extra fields.

UNIONS

Unions: “one of these types”, with no shared fields required

```
schema

union SearchResult = Book | Author | Publisher

type Query {
  search(text: String!): [SearchResult!]!
}
```

```
query

query {
  search(text: "ada") {
    __typename
    ... on Book { title }
    ... on Author { name }
  }
}
```

- 01** Members must be **object** types — not scalars, interfaces or other unions.
- 02** You cannot select any field directly on a union except **__typename**; every other field needs a fragment.
- 03** Unlisted members (Publisher above) simply return only **__typename**.

CHOOSING

Interface or union?

Question	Interface	Union
Do the types share fields?	Yes — that is the point	Not required
Select shared fields without fragments?	Yes	No (only __typename)
Can members be interfaces?	Types implement it; interfaces can too	Members are objects only
Typical use	Node, Timestamped, Publication	Search results, mutation results, feed items of unrelated kinds
Adding a new type later	The new type must implement all fields	Add it to the member list

Both are **abstract types**: the server must tell which concrete type an object is — via `__resolveType` or an `isTypeOf` check (Section 09).

DESCRIPTIONS

Descriptions turn your schema into its own documentation

```
"""
An individual reader review.
Stars are 1–5; body is optional Markdown.
"""

type Review {
  "Stable unique identifier."
  id: ID!
  "Rating from 1 to 5."
  stars: Int!
  "Optional review text, in Markdown."
  body: String
}
```

- 01 Block strings """...""" or single-line "..." placed **before** a definition.
- 02 Descriptions are exposed via introspection, so GraphQL, Apollo Studio and codegen display them.
- 03 CommonMark (Markdown) is the conventional format.
- 04 Document units, ranges, nullability reasons and side effects.

CUSTOM DIRECTIVES

Declaring your own directives

```
enum Role { READER EDITOR ADMIN }

directive @auth(requires: Role = ADMIN)
  on OBJECT | FIELD_DEFINITION

directive @tag(name: String!)
  repeatable on FIELD_DEFINITION

type Book {
  title: String!
  costPrice: Float @auth(requires: EDITOR)
  sales: Int @tag(name: "internal") @tag(name: "bi")
}
```

- 01 **on** lists the locations where it may appear.
- 02 **repeatable** allows several on one location.
- 03 Declaring a directive only adds syntax — the **behaviour** is implemented by the server or a tool, for example via schema transforms.

EXTENSIONS

Extending types lets you split a schema into modules

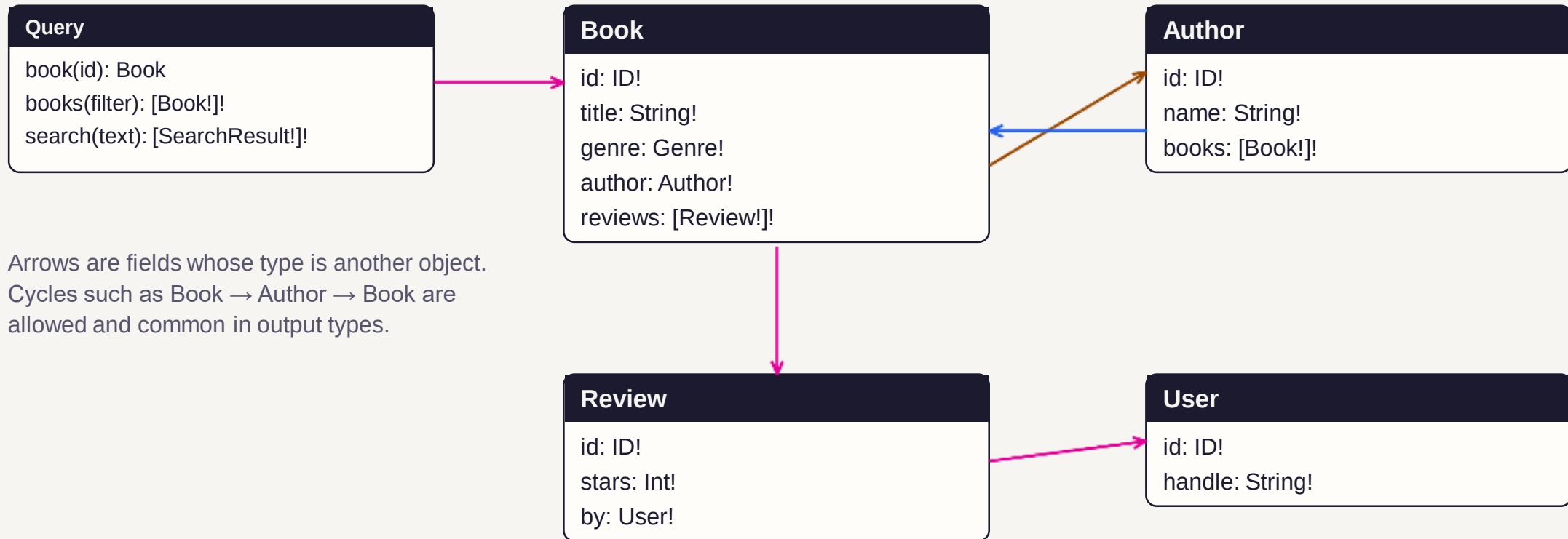
```
# books.graphql
type Book { id: ID! title: String! }
type Query { book(id: ID!): Book }

# reviews.graphql
type Review { id: ID! stars: Int! }
extend type Book {
  reviews: [Review!]!
}
extend type Query {
  recentReviews: [Review!]!
}
```

- 01 `extend type`, `extend interface`, `extend enum`, `extend input`, `extend union`, `extend scalar` and `extend schema` all exist.
- 02 Each feature team owns a file; the server merges them into one schema at start-up.
- 03 The same idea, applied across services, becomes **federation** (Section 15).

PUTTING IT TOGETHER

The bookstore schema drawn as a graph



CHECKPOINT 07

Test yourself before moving on

Q1. What is the difference between `[Book!]` and `[Book]!`?

Q2. Where does a null in a non-null field end up?

Q3. Can a union have an interface as a member?

Q4. Why can't an object type be used as an argument type?

Answers are in the speaker notes.

SECTION 08 · INTERMEDIATE

Introspection & tooling

A GraphQL server can answer questions about its own schema. That single feature powers autocomplete, documentation, code generation and more.

IN THIS SECTION

- 01 The introspection system
- 02 Querying `__schema`
- 03 Querying `__type` and unwrapping types
- 04 What introspection powers
- 05 Introspection in production
- 06 The tooling landscape

The schema can be queried with GraphQL itself

```
query SchemaOverview {  
  __schema {  
    queryType { name }  
    mutationType { name }  
    types {  
      name  
      kind  
      description  
    }  
    directives { name locations }  
  }  
}
```

1

1. __schema

A root meta-field available on every server.

2

3

2. Root operation types

Names of Query, Mutation, Subscription.

4

3. Every named type

Including built-ins such as String and __Type.

4. kind

SCALAR, OBJECT, INTERFACE, UNION, ENUM, INPUT_OBJECT, LIST or NON_NULL.

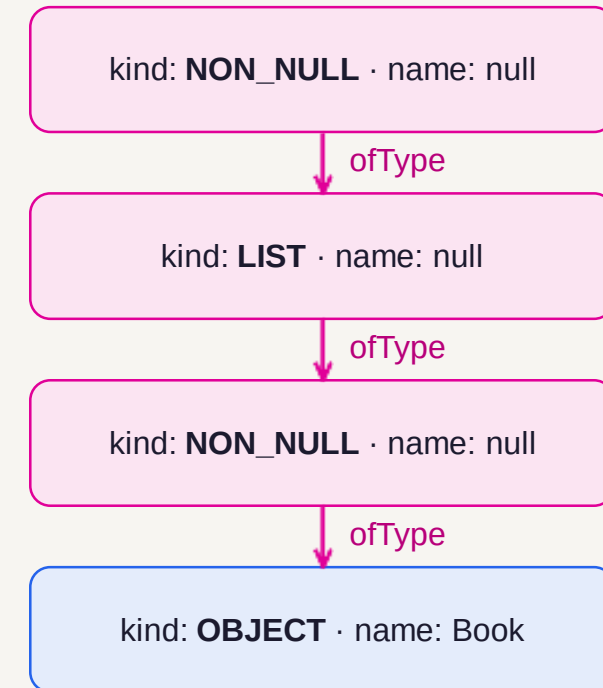
INTROSPECTION

__type and the ofType chain unwrap lists and non-nulls

```
query {  
  __type(name: "Author") {  
    fields {  
      name  
      type { kind name  
        ofType { kind name  
          ofType { kind name  
            ofType { kind name } } }  
        }  
      }  
    }  
  }  
}
```

Wrapping types have no name; you follow **ofType** until you reach a named type. That is why generated introspection queries nest **ofType** several levels deep.

How books: [Book!]! is described:



TOOLING

One machine-readable schema, a whole ecosystem of tools



Because the type system is machine-readable, every tool in the ecosystem works with every compliant server — no per-API plugins.

Should introspection be enabled in production?



Public APIs: usually yes

Third-party developers need the schema to build against it. GitHub and Shopify, for example, publish public schemas.



Private APIs: often disabled

Your own clients get the schema at build time. Disabling introspection hides your attack surface map — but it is **not** a security control on its own.

HIDDEN LEAK

Field suggestions in error messages (“Did you mean...”) can leak schema details even with introspection off. Disable those too if the schema is secret, and publish it to a private **schema registry** for internal tools.

TOOLING

The tooling landscape at a glance

Category	Examples	What it does
IDEs	GraphiQL, Apollo Sandbox, Altair, Insomnia, Postman	Autocomplete, docs, run operations
Code generation	GraphQL Code Generator, Relay Compiler, Apollo iOS/Kotlin codegen	Types and hooks from schema + operations
Schema quality	graphql-eslint, GraphQL Inspector	Linting, breaking-change detection
Registries	Apollo GraphOS, Hive, WunderGraph Cosmo	Versioned schemas, usage metrics, composition
Visualisation	GraphQL Voyager	Interactive graph of types
Mocking	graphql-tools mocks, MSW GraphQL handlers	Fake data for UI and tests

Product names change often; evaluate current options when you adopt one.

CHECKPOINT 08

Test yourself before moving on

Q1. Which meta-fields make up introspection?

Q2. How do you get from [Book!]! to the name Book in introspection?

Q3. Is disabling introspection a security control?

Q4. Name three tools powered by introspection.

Answers are in the speaker notes.

SECTION 09 · INTERMEDIATE

Execution & resolvers

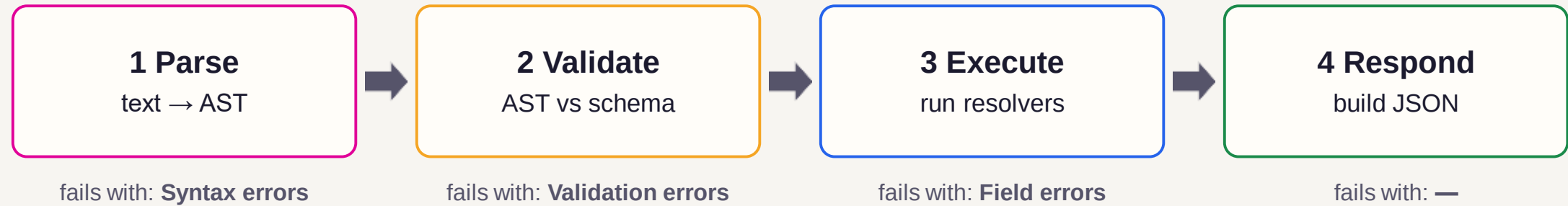
Open the engine. Follow a request from raw text to JSON: parsing, validation, the resolver chain, value completion and error handling.

IN THIS SECTION

- 01 The three phases
- 02 Parsing to an AST
- 03 Validation rules
- 04 Resolvers and their signature
- 05 Default resolvers
- 06 Walking the resolver chain
- 07 Value completion
- 08 Abstract types
- 09 Execution errors
- 10 The info object

THE PIPELINE

Every request passes through three phases



Phases 1–2 are static. They depend only on the document and the schema, so their result can be cached per document. A failure means no resolver runs and the response contains only **errors**.

Phase 3 is dynamic. It depends on data. A failure in one field produces an error entry and a null, while other fields still return data: **partial success**.

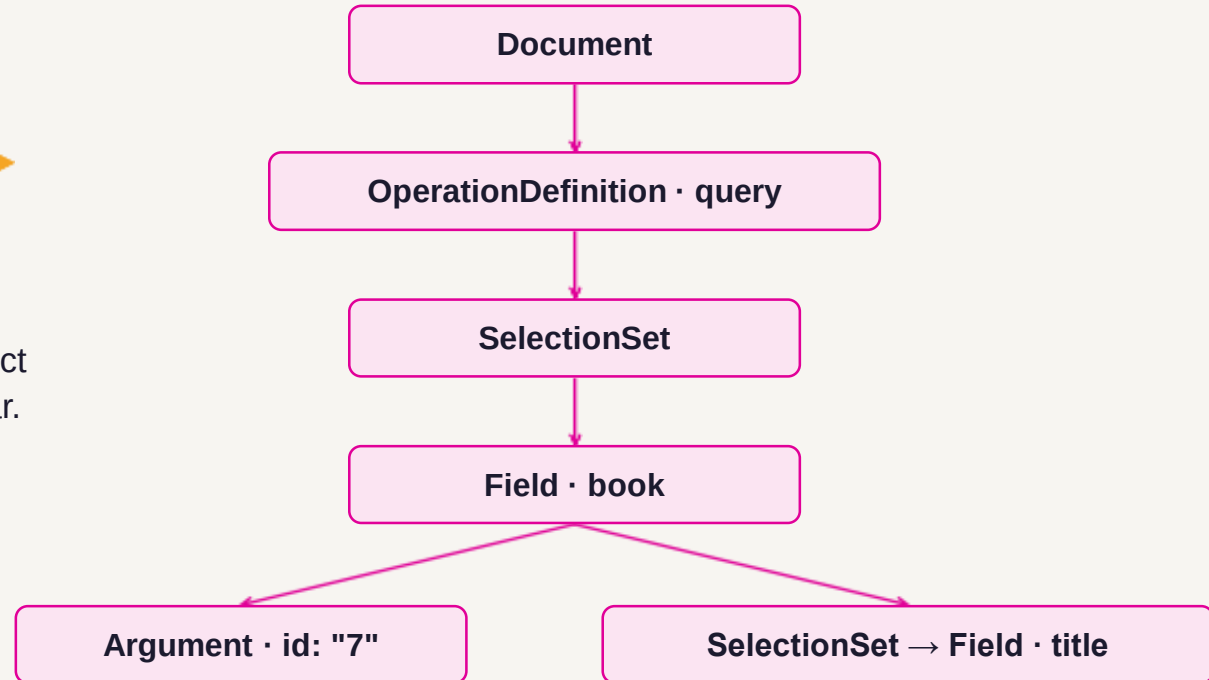
PHASE 1 · PARSE

Parsing turns the document into an abstract syntax tree

```
query {  
  book(id: "7") {  
    title  
  }  
}
```



The **lexer** splits text into tokens; the **parser** builds an Abstract Syntax Tree. Whitespace, commas and comments disappear. Every node records its line and column — which is where error **locations** come from.



Validation proves the document can run against this schema

Rule (spec name, abridged)	Catches
Fields on correct type	Selecting a field that does not exist on that type
Leaf field selections	Missing or extra { } on objects and scalars
Argument names / required arguments	Unknown or missing arguments
Values of correct type	first: "ten" for an Int
Fields in set can merge	Two fields with the same response key but different arguments
Fragments: unique, used, no cycles, possible spreads	The fragment rules from Section 04
Variables: unique, defined, used, allowed in position	The variable rules from Section 03
Operation name uniqueness / lone anonymous operation	Ambiguous documents
Single root field (subscriptions)	Subscriptions selecting several roots

RESOLVERS

A resolver is a function that produces one field's value

```
const resolvers = {  
  Query: {  
    book: (_parent, args, ctx) =>  
      ctx.db.books.findById(args.id),  
  },  
  Book: {  
    author: (book, _args, ctx) =>  
      ctx.db.authors.findById(book.authorId),  
    reviewCount: (book, _args, ctx) =>  
      ctx.db.reviews.count({ bookId: book.id }),  
  },  
}
```

- 01** Resolvers are organised as **TypeName** → **fieldName** → **function**.
- 02** Each may return a value, a Promise, or (for lists) an array of values or Promises.
- 03** The value returned by a parent's resolver becomes the **parent** argument of its children's resolvers.
- 04** Resolvers can fetch from any source: SQL, REST, gRPC, cache, file, computation.

RESOLVERS

The four arguments every resolver receives

```
Book: {  
  author(  
    parent, // the Book object  
    args,   // { } this field's args  
    context, // per-request shared  
    info    // AST + schema details  
  ) {  
    return context.loaders.author  
      .load(parent.authorId)  
  }  
}
```

-
- 1. parent (or root / obj)**
The result of the parent field's resolver.
 - 2. context**
Created once per request: user, loaders, DB.
 - 3. info**
Field name, path, return type, the AST.
 - 4. Return a value or Promise**
The engine awaits it and completes it.

RESOLVERS

Most fields need no resolver at all

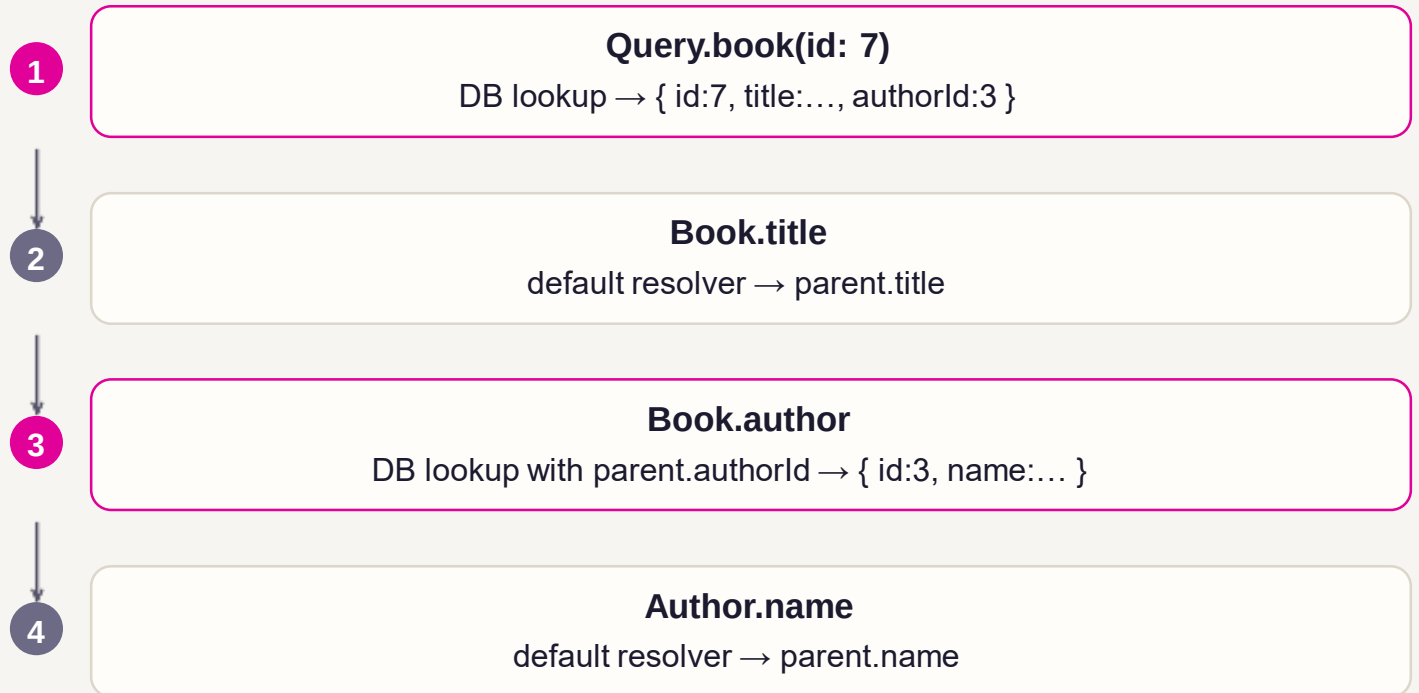
```
// No resolver written for Book.title:  
// the engine uses the default resolver  
function defaultResolver(parent, args, ctx, info) {  
  const v = parent[info.fieldName]  
  return typeof v === "function"  
    ? v.call(parent, args, ctx, info)  
    : v  
}
```

- 01 If the parent object has a property with the field's name, the default resolver returns it.
- 02 So shape your data objects to match the schema where you can, and write resolvers only for computed or related fields.
- 03 Mismatched names (**author_id** vs **authorId**) need a one-line resolver — or a mapping layer.

WALK-THROUGH

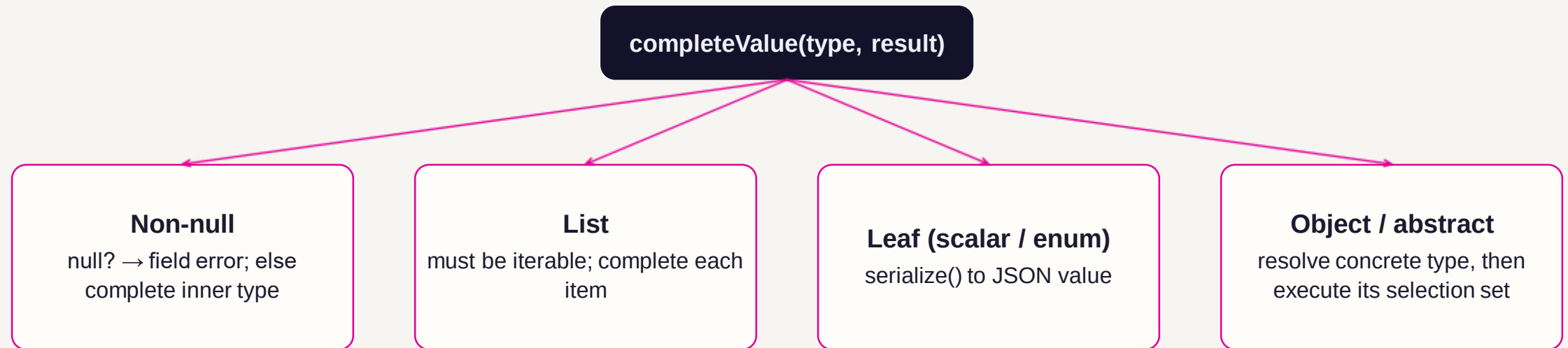
Resolving a query, one field at a time

```
{
  book(id: "7") {
    title
    author {
      name
    }
  }
}
```



Parents always resolve before their children; siblings (**title** and **author**) are independent and may run concurrently.

Completion turns raw resolver results into schema-correct values



After a resolver returns, the engine **completes** the value according to the field's declared type. This is where **!** promises are enforced, lists are mapped item by item, scalars are serialised and the recursion into sub-selections happens. Resolvers produce raw data; completion guarantees the schema's contract.

ABSTRACT TYPES

Interfaces and unions need a way to name the concrete type

```
const resolvers = {  
  SearchResult: {  
    __resolveType(obj) {  
      if ("isbn" in obj) return "Book"  
      if ("bornIn" in obj) return "Author"  
      return null // → execution error  
    },  
  },  
}  
  
// alternative: per object type  
Book: { __isTypeOf: (obj) => "isbn" in obj }
```

- 01 When a field returns an interface or union, the engine must decide which object type each value is.
- 02 `__resolveType` on the abstract type returns the type name.
- 03 Tip: include a discriminator (**kind** or `__typename`) in your data objects so this is a simple lookup, not guesswork.

EXECUTION ERRORS

A resolver error nulls its field and adds an error entry

```
Book: {  
  price: async (book, _, ctx) => {  
    const p = await ctx.pricing.get(book.id)  
    if (!p) throw new GraphQLError(  
      "Pricing unavailable",  
      { extensions: { code: "PRICING_DOWN" } })  
    return p.amount  
  },  
}
```

```
{  
  "data": { "book": {  
    "title": "Notes",  
    "price": null } },  
  "errors": [{  
    "message": "Pricing unavailable",  
    "path": ["book", "price"],  
    "locations": [{"line": 4, "column": 5}],  
    "extensions": { "code": "PRICING_DOWN" } }]  
}
```

price is nullable, so only it becomes null. Had it been **Float!**, null propagation would have nulled **book** instead.

EXECUTION ERRORS

Classify, mask and log errors deliberately



Classify

Put a stable machine-readable **extensions.code** on every error you throw: **UNAUTHENTICATED**, **FORBIDDEN**, **NOT_FOUND**, **BAD_USER_INPUT**.



Mask

Unexpected exceptions can leak SQL, stack traces or hostnames. Replace their message with a generic one before sending; many servers can do this for you.



Log with context

Log the full error server-side with operation name, path and a request ID that you also return in **extensions**.

The **code** values above are widespread conventions, not part of the GraphQL specification. Agree on a list for your organisation and document it.

THE INFO OBJECT

info: everything the engine knows about the current field

Property (graphql-js)	Contains	Typical use
fieldName	Name of the field being resolved	Generic resolvers, logging
path	Response path, e.g. book → reviews → 2	Error reporting, tracing
returnType / parentType	Schema types involved	Generic middleware
fieldNodes	AST nodes of this field, with its sub-selection	Look-ahead: which sub-fields were requested?
variableValues	Coerced variables of the operation	Custom directives
schema, operation, fragments	The whole picture	Advanced tooling, cost analysis

Look-ahead lets a resolver fetch only the columns requested, or join related tables in one SQL query — covered in Section 11.

CHECKPOINT 09

Test yourself before moving on

Q1. In which phase is a misspelled field name caught?

Q2. What does the default resolver return?

Q3. A resolver for a nullable field throws. What does the client get?

Q4. What is `__resolveType` for?

Answers are in the speaker notes.

SECTION 10 · INTERMEDIATE

Building a server

Turn the theory into a running API. We build the bookstore server twice — schema-first in TypeScript and code-first in Python — and structure it for growth.

IN THIS SECTION

- 01 The server ecosystem
- 02 Schema-first vs code-first
- 03 A schema-first server in Node.js
- 04 A code-first server in Python
- 05 Project structure
- 06 Per-request context
- 07 Authentication
- 08 Layered architecture
- 09 Mutation resolvers
- 10 Testing

ECOSYSTEM

Mature GraphQL servers exist for every major language

Language	Popular libraries	Style
JavaScript / TypeScript	graphql-js (reference), Apollo Server, GraphQL Yoga, Mercurius, Pothos, NestJS GraphQL	Both
Python	Strawberry, Graphene, Ariadne	Code-first (Strawberry, Graphene), schema-first (Ariadne)
Java / Kotlin	graphql-java, Spring for GraphQL, DGS Framework, graphql-kotlin	Both
Go	gqlgen, graphql-go	gqlgen generates code from the schema
C# / .NET	Hot Chocolate, GraphQL.NET	Both
Rust	async-graphql, Juniper	Code-first
Ruby	graphql-ruby	Code-first

APPROACH

Schema-first vs code-first

Schema-first — write SDL, then resolvers

```
type Book {  
  id: ID!  
  title: String!  
}
```

- 01 SDL is the design artefact; easy to review
- 02 Needs codegen to keep resolver types in sync

Code-first — write types in code, SDL is generated

```
@strawberry.type  
class Book:  
    id: strawberry.ID  
    title: str
```

- 01 One source of truth; the compiler checks resolvers
- 02 The schema's shape is spread across code

Both produce the same runtime schema. Choose by team preference and language idiom; many teams start schema-first to design, then generate types.

A complete server in about twenty lines

```
import { createServer } from "node:http"
import { createSchema, createYoga } from "graphql-yoga"

const typeDefs = /* GraphQL */ `
  type Book { id: ID! title: String! year: Int }
  type Query { book(id: ID!): Book books: [Book!]! }
`

const BOOKS = [{ id: "7", title: "Notes", year: 1843 }]

const resolvers = {
  Query: {
    book: (_, { id }) => { id: string }) =>
      BOOKS.find((b) => b.id === id) ?? null,
    books: () => BOOKS,
  },
}

const yoga = createYoga({ schema: createSchema({ typeDefs, resolvers }) })
createServer(yoga).listen(4000) // → http://localhost:4000/graphql
```

- 01** **createSchema** joins SDL and resolvers into an executable schema.
- 02** **createYoga** handles HTTP, GraphQL and errors.
- 03** Swap in Apollo Server or Mercurius with the same **typeDefs** and **resolvers**.

The same API with Strawberry and FastAPI

```
import strawberry
from fastapi import FastAPI
from strawberry.fastapi import GraphQLRouter

@strawberry.type
class Book:
    id: strawberry.ID
    title: str
    year: int | None = None

BOOKS = {"7": Book(id="7", title="Notes", year=1843)}

@strawberry.type
class Query:
    @strawberry.field
    def book(self, id: strawberry.ID) -> Book | None:
        return BOOKS.get(id)

schema = strawberry.Schema(query=Query)
app = FastAPI()
app.include_router(GraphQLRouter(schema), prefix="/graphql")
```

- 01 Python type hints become GraphQL types: `str` → `String!`, `int | None` → `Int`.
- 02 Methods decorated with `@strawberry.field` are resolvers.
- 03 Run with `uvicorn main:app`; GraphQL is served at `/graphql`.

STRUCTURE

A project layout that scales with the schema

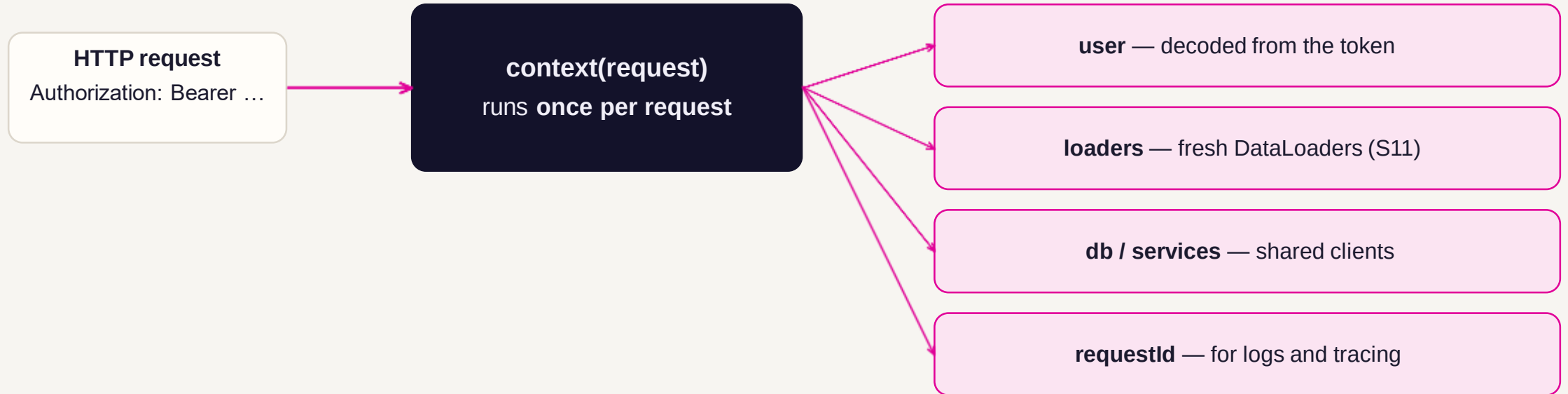
```
src/
├── schema/
│   ├── books.graphql    # SDL per domain
│   ├── reviews.graphql
│   └── users.graphql
├── resolvers/
│   ├── book.ts          # thin: map fields → services
│   ├── review.ts
│   └── index.ts          # merge resolver maps
├── services/             # business rules, auth checks
├── data/                 # repositories, SQL, REST clients
├── loaders/              # DataLoader factories (S11)
├── context.ts            # build per-request context
└── server.ts             # HTTP, plugins, limits
```

Organise by **domain** (books, reviews, users), not by technical kind, once the schema grows past a handful of types.

Keep resolvers **thin**. They translate GraphQL into calls on services; the rules live in services so REST, jobs and GraphQL share them.

CONTEXT

Context carries per-request state to every resolver



Every resolver in the request receives the same context object as its third argument. Put **request-scoped** things there — never store per-user data in module-level variables, which are shared by all concurrent requests.

AUTHENTICATION

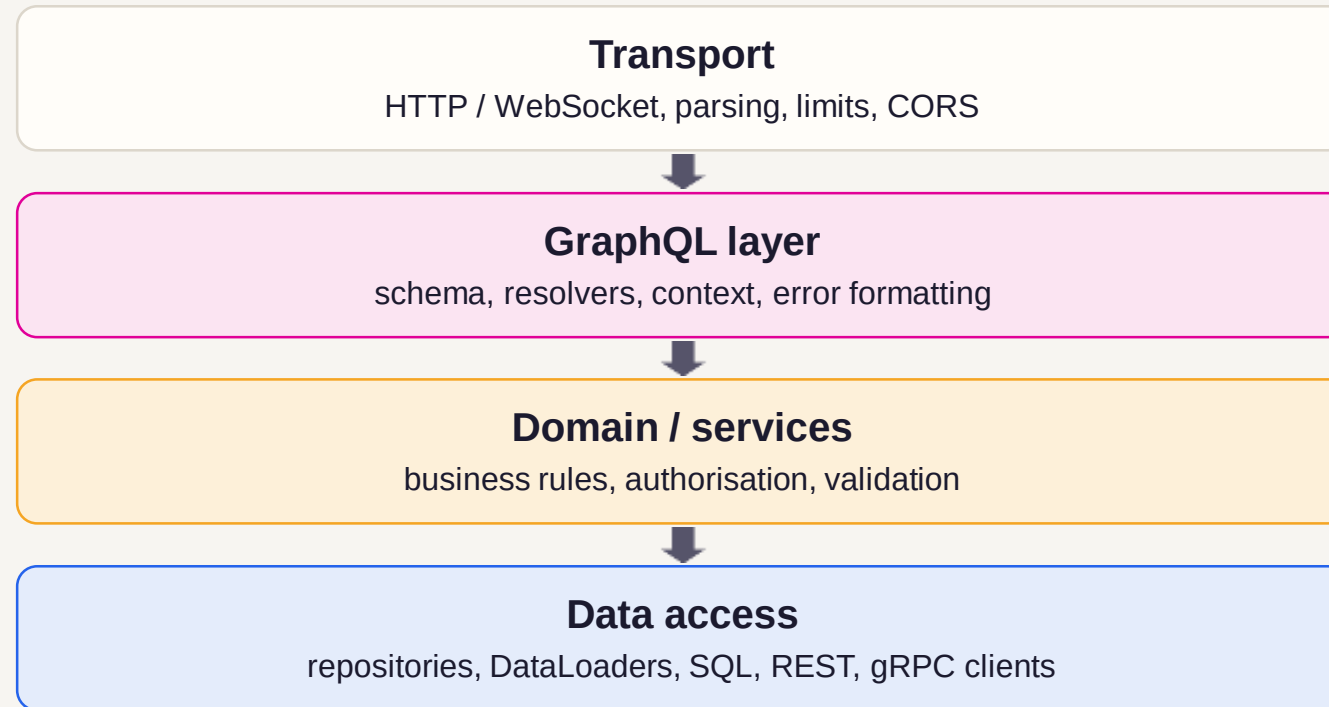
Authenticate once in context; authorise in resolvers or services

```
const yoga = createYoga({
  schema,
  context: async ({ request }) => {
    const header = request.headers.get("authorization")
    const token = header?.replace("Bearer ", "")
    const user = token ? await verifyJwt(token) : null
    return { user, loaders: makeLoaders(db), db }
  },
})

// in a resolver or service
function requireUser(ctx) {
  if (!ctx.user) throw new GraphQLError("Sign in required",
    { extensions: { code: "UNAUTHENTICATED" } })
  return ctx.user
}
```

- 01 Authentication** (who are you?) happens once, at the edge.
- 02** Let anonymous requests through with **user: null**; public fields still work.
- 03 Authorisation** (may you see this?) belongs close to the data — Section 12.

Treat GraphQL as a thin layer over your domain



Why layers? The GraphQL layer is an adapter. If authorisation lives in resolvers only, a new REST endpoint or background job can bypass it.

Rule: resolvers call services; services call data access. Nothing below the GraphQL layer knows GraphQL exists.

MUTATION RESOLVERS

A mutation resolver: validate, authorise, write, return a payload

```
addReview: async (_p, { input }, ctx) => {  
  const user = requireUser(ctx)           // 1. authenticate  
  const errors = []  
  if (input.stars < 1 || input.stars > 5)   // 2. validate  
    errors.push({ field: ["input", "stars"], message: "Stars must be 1-5", code: "OUT_OF_RANGE" })  
  if (await ctx.db.reviews.exists(user.id, input.bookId)) // 3. business rule  
    errors.push({ field: ["input", "bookId"], message: "Already reviewed", code: "DUPLICATE" })  
  if (errors.length) return { review: null, book: null, userErrors: errors }  
  
  const review = await ctx.db.reviews.insert({ ...input, userId: user.id }) // 4. write  
  ctx.pubsub.publish(`REVIEW:${input.bookId}`, { review })           // 5. notify  
  const book = await ctx.loaders.book.load(input.bookId)  
  return { review, book, userErrors: [] }                             // 6. payload  
}
```

TESTING

Test the schema by executing real operations

```
import { graphql } from "graphql"
import { schema } from "../src/schema"

test("book returns title", async () => {
  const result = await graphql({
    schema,
    source: `query($id: ID!) { book(id: $id) { title } }`,
    variableValues: { id: "7" },
    contextValue: makeTestContext({ user: null }),
  })
  expect(result.errors).toBeUndefined()
  expect(result.data).toEqual({ book: { title: "Notes" } })
})
```

- 01** Run operations through the **real** schema: validation, resolvers and null handling are all exercised.
- 02** Inject a test context with fakes for the DB and a chosen user.
- 03** Unit-test services separately; keep resolver tests at operation level.

CHECKPOINT 10

Test yourself before moving on

Q1. What is the main trade-off between schema-first and code-first?

Q2. Why must DataLoaders and the current user live in context, not module scope?

Q3. Where should authorisation rules live?

Q4. How do you test a resolver realistically?

Answers are in the speaker notes.

SECTION 11 · INTERMEDIATE

Performance & DataLoader

Field-by-field resolution is elegant — and, done naively, dangerously slow. Learn the N+1 problem and the standard tools that solve it.

IN THIS SECTION

- 01 The N+1 problem
- 02 Seeing it in the logs
- 03 How DataLoader batches
- 04 Writing a batch function
- 05 Before and after
- 06 Per-request loaders
- 07 Look-ahead and joins
- 08 Caching layers
- 09 Persisted queries

THE N+1 PROBLEM

Naive resolvers issue one query per item in a list

```
query {  
  books(first: 50) {  
    title  
    author {  
      name  
    }  
  }  
}
```

Book.author runs once **per book**. Each call fetches one author independently.

1 query — SELECT * FROM books LIMIT 50

SELECT * FROM authors WHERE id = 3

SELECT * FROM authors WHERE id = 8

SELECT * FROM authors WHERE id = 3

SELECT * FROM authors WHERE id = 11

... **50 more queries**, one per book — many for the same author.

N+1: 1 query for the list + N queries for its children. Nest one more level (author → books → reviews) and it multiplies.

THE N+1 PROBLEM

What N+1 looks like in the database log

```
-- one GraphQL request
SELECT * FROM books LIMIT 50;
SELECT * FROM authors WHERE id = 3;
SELECT * FROM authors WHERE id = 8;
SELECT * FROM authors WHERE id = 3; -- duplicate
SELECT * FROM authors WHERE id = 11;
SELECT * FROM authors WHERE id = 8; -- duplicate
-- ... 45 more lines
```

Two separate waste sources:

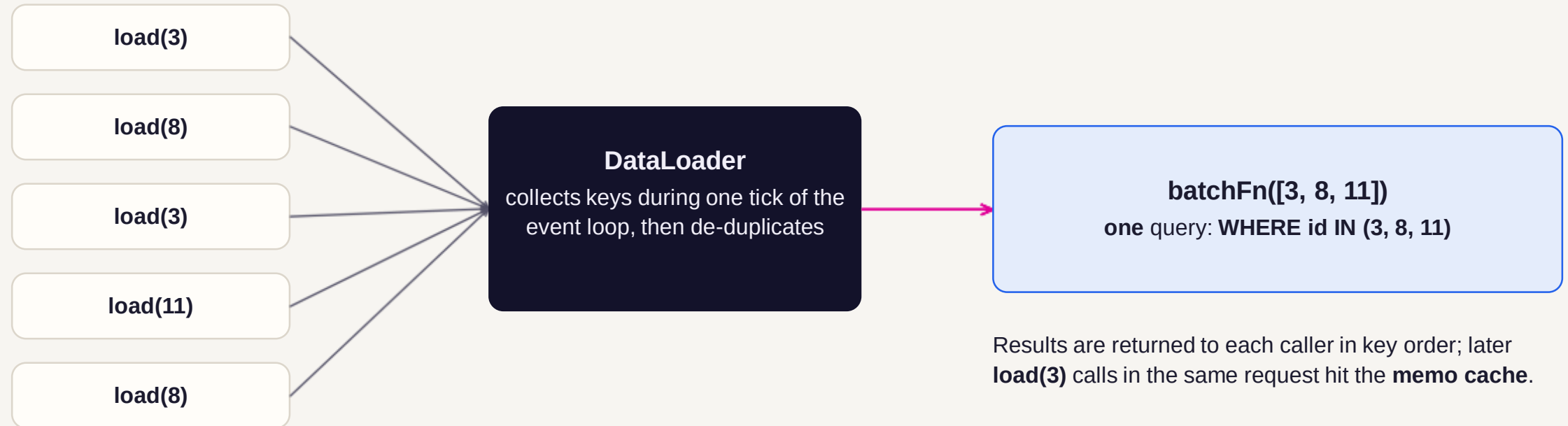
- 01 Too many round trips** — each with network latency and query overhead.
- 02 Duplicate work** — the same author fetched repeatedly.

CONSTRAINT

The fix must work without the resolvers knowing about each other — each one only knows its own parent.

DATALOADER

DataLoader: batch and de-duplicate loads within a request



Batching fixes the round trips. Caching fixes the duplicates. Together: $N+1$ becomes $1+1$.

DATALOADER

Writing a loader and its batch function

```
import DataLoader from "dataloader"

export const makeLoaders = (db) => ({
  author: new DataLoader(async (ids) => {
    const rows = await db.query(
      "SELECT * FROM authors WHERE id = ANY($1)", [ids]
    )
    const byId = new Map(rows.map((r) => [r.id, r]))
    return ids.map((id) => byId.get(id) ?? null)
  }),
})

// resolver
Book: {
  author: (book, _a, ctx) =>
    ctx.loaders.author.load(book.authorId),
}
```

THE CONTRACT

The batch function must return an array of the **same length** as **ids**, in the **same order**.

Databases return rows in any order — always re-map by key.

Missing keys map to **null** (or an **Error** for that key), never to a shorter array.

Other languages have ports: `aiodataloader` and Strawberry's `DataLoader` in Python, `java-dataloader`, `GreenDonut` in .NET.

IMPACT

Before and after batching

Request shape	Naive resolvers	With DataLoader
50 books → author	1 + 50 queries	2 queries
50 books → author → 10 books each	1 + 50 + 50 queries	3 queries
50 books → reviews (5 each) → reviewer	1 + 50 + 250 queries	3 queries

Counts assume one loader per relationship and no duplicates. The key property: query count grows with the **depth** of the query, not the **number of items**.

Measure in your own stack: add a query counter to your test context and assert on it, so an N+1 regression fails CI.

Create loaders per request — never share them



Why not a global loader?

Its memo cache would live forever: stale data across requests, unbounded memory growth.



The security reason

User A's request could populate a cache that user B's request then reads — bypassing authorisation on the data source.



The right pattern

Build fresh loaders in the context factory for each request, as in **makeLoaders(db)** above.

DataLoader's cache is a **request-scoped memo**, not an application cache. For cross-request caching, use Redis or HTTP caching deliberately, with keys that include authorisation scope.

LOOK-AHEAD

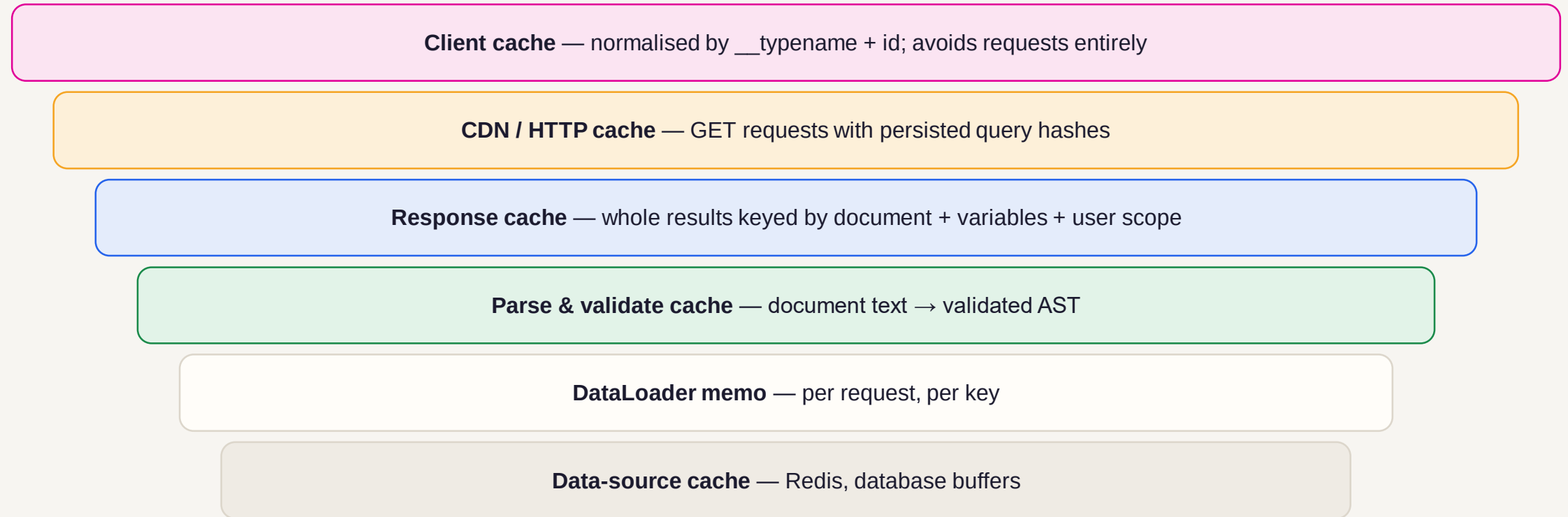
Look-ahead: plan data access from the whole selection

```
Query: {  
  books: async (_p, args, ctx, info) => {  
    const want = requestedFields(info) // from fieldNodes  
    const cols = ["id", ...want.scalars]  
    const joinAuthor = want.has("author")  
    return ctx.db.books.find({  
      limit: args.first, columns: cols,  
      include: joinAuthor ? ["author"] : [] })  
  },  
}
```

- 01 Inspect **info** to see which sub-fields were selected, then select only those columns or join related tables up front.
- 02 Libraries help: graphql-parse-resolve-info (JS), Join Monster, PostGraphile and Hasura compile whole queries to SQL.
- 03 Trade-off: faster, but resolvers become coupled to the query shape. Use for hot paths, DataLoader everywhere else.

CACHING

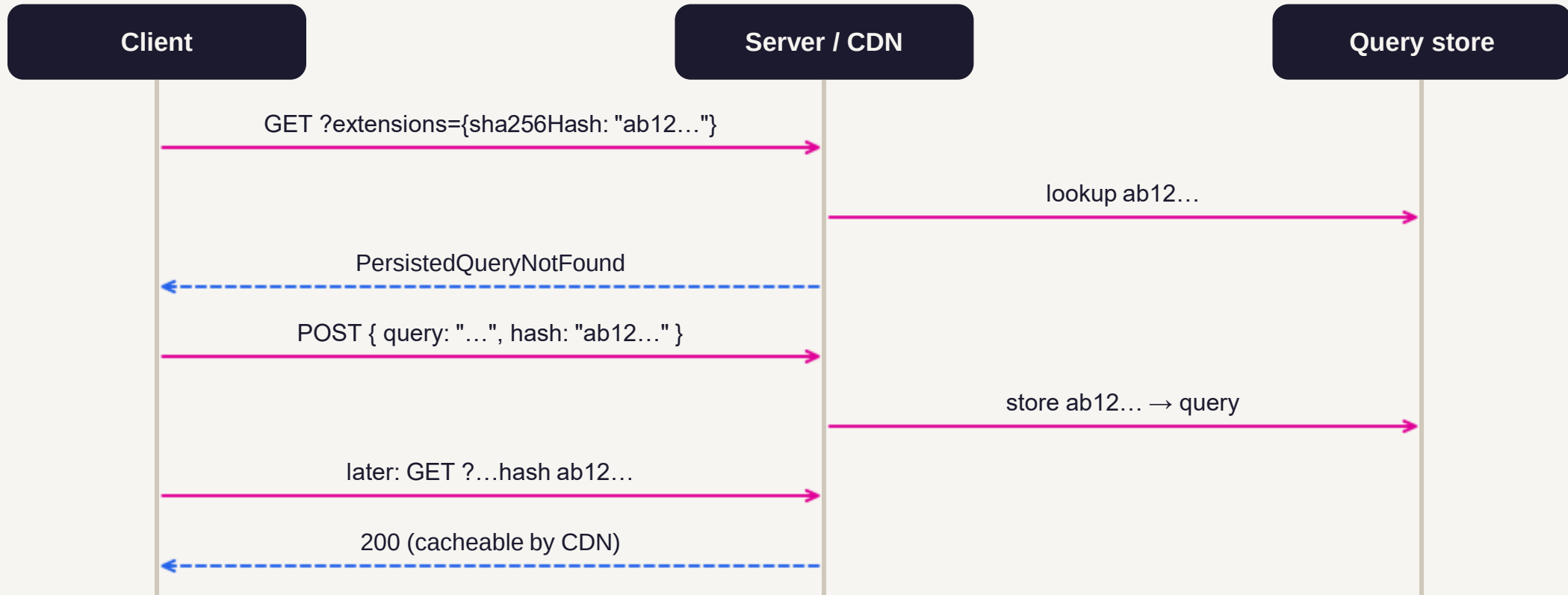
Six caching layers, from browser to database



Closest to the user (top) saves the most work; each lower layer catches what the one above missed.

PERSISTED QUERIES

Automatic persisted queries: send a hash instead of the text



More techniques worth knowing



Limit list sizes

Require **first/last** on connections with a maximum (say 100). Unbounded lists are both slow and an attack vector.



Defer slow fields

@defer lets fast parts of a response arrive first while expensive fragments stream in later (Section 15).



Trace per resolver

Measure time per field path. The slowest resolver in a query is rarely the one you guessed.



Cache parsed documents

Parsing and validation are pure functions of the document text: cache them, especially with persisted queries.



Avoid resolver waterfalls

Start independent async work in parallel; do not **await** sequentially inside one resolver.



Denormalise hot paths

A stored **reviewCount** beats counting rows on every request.

CHECKPOINT 11

Test yourself before moving on

Q1. Why does N+1 happen in GraphQL?

Q2. What two things does DataLoader do?

Q3. What must a batch function return?

Q4. Why must DataLoaders be per request?

Answers are in the speaker notes.

SECTION 12 · ADVANCED

Security

A single endpoint that accepts arbitrary queries is powerful for clients — and for attackers. Build defences in layers.

IN THIS SECTION

- 01 The GraphQL threat model
- 02 Deeply nested queries
- 03 Depth limiting
- 04 Cost and complexity analysis
- 05 Aliases and batching abuse
- 06 Rate limiting by cost
- 07 Trusted documents
- 08 Authorisation patterns
- 09 Input validation and error hygiene
- 10 A security checklist

THREAT MODEL

What changes when clients write the queries

Threat	How it looks in GraphQL	Primary defence
Resource exhaustion	Deep or wide queries that explode into millions of resolver calls	Depth, cost and list-size limits; timeouts
Brute force via batching	Hundreds of login attempts aliased into one request	Per-operation rate limits; limit aliases
Data exposure	Fields reachable through unexpected paths in the graph	Authorisation at the data or service layer
Schema reconnaissance	Introspection and field suggestions map the attack surface	Disable in private APIs; trusted documents
Injection	User input concatenated into SQL or into documents	Variables; parameterised data access
Information leakage	Stack traces or SQL in error messages	Error masking

RESOURCE EXHAUSTION

A short, valid query can demand enormous work

```
query Evil {  
  author(id: "1") {  
    books {  
      author {  
        books {  
          author {  
            books {  
              reviews { body }  
            }  
          }  
        }  
      }  
    }  
  }  
}
```

1

1. A legal cycle

Author → Book → Author is valid schema design.

2

2. Each level multiplies

50 books × 50 books × ... grows geometrically.

3

3. Validation passes

The document is well-formed and type-correct.

Nothing in the core spec limits query size. You must.

DEPTH LIMITING

Depth limiting: the simplest first line of defence

```
import depthLimit from "graphql-depth-limit"
```

```
const server = new ApolloServer({  
  schema,  
  validationRules: [depthLimit(8)],  
})
```

Implemented as an extra **validation rule**, so offending documents are rejected before execution, at almost no cost.

Choosing the limit: measure the deepest legitimate operation your clients send, then add a small margin.

Its weakness: depth ignores width. **books(first: 1000) { reviews(first: 1000) { ... } }** is only depth 3 but can mean a million objects.

So depth limiting is necessary but not sufficient — pair it with cost analysis.

COST ANALYSIS

Cost analysis: estimate the work before doing it

```
query {                                # cost
  books(first: 50) {                  # 50
    title                            # 0
    author { name }                  # 50
    reviews(first: 10) {             # 500
      body                            # 0
    }
  }
}
```

One simple model shown: each object costs 1, multiplied by the size of every enclosing list. Real systems tune weights per field.

Estimate before executing

Walk the validated AST, multiply by list sizes from **first** / **last** arguments.

Weight fields

Scalars ≈ 0 ; objects ≈ 1 ; expensive fields (search, external APIs) get explicit weights.

Reject or throttle

Refuse documents above a maximum; charge the cost against the client's budget.

BATCHING ABUSE

Aliases and batching can multiply one operation into hundreds

```
mutation {  
  a1: login(user: "ada", pass: "aaaa") { token }  
  a2: login(user: "ada", pass: "aaab") { token }  
  a3: login(user: "ada", pass: "aaac") { token }  
  # ... a1000  
}
```

- 01** One HTTP request can bypass a “5 requests per minute” limit on the login endpoint.
- 02** The same trick works with array-batched requests (a JSON array of operations) where supported.
- 03** **Defences:** cap aliases per document, cap operations per batch, and rate-limit **per resolver call** for sensitive fields.

RATE LIMITING

Rate-limit by cost, not by request count



Request counting

100 requests / minute. Fails in GraphQL: one request may cost 1 point or 1,000,000.



Cost budgets

e.g. [5,000] points / hour per client. Each query's calculated cost is deducted; expensive queries consume more of the budget.



Field-level limits

Extra per-field limits for sensitive actions: login, password reset, sending emails.

Return the budget state to clients — for example remaining points and reset time in **extensions** or headers — so well-behaved clients can pace themselves.

TRUSTED DOCUMENTS

The strongest control: only run operations you shipped



Trusted documents (also called persisted operations or operation allow-listing) flip the model: the server accepts only operations your own clients were built with. Attackers can no longer craft deep, wide or reconnaissance queries at all.

Best for first-party APIs where you ship every client. Public APIs for third parties cannot use it, and rely on cost analysis instead.

AUTHORISATION

Where authorisation checks can live

Layer	Example	Strength	Risk
Schema directive	costPrice: Float @auth(requires: EDITOR)	Visible in the schema, declarative	Only covers fields you remember to annotate
Resolver	if (!canView(ctx.user, book)) return null	Precise, has the parent object	Duplicated logic; easy to miss one path
Service / domain	bookService.get(id, user) enforces rules	One rule for every API and job	Needs the user passed down
Data layer	Row-level security in the database	Impossible to bypass from code	Harder to express business rules

GRAPH-SPECIFIC TRAP

In a graph, the same object is reachable by many paths (**book(id)**, **author.books**, **search**). Check permissions where the **object** is loaded, not where the **path** starts.

Input validation and error hygiene



Validate inputs

Types check shape, not meaning. Enforce lengths, ranges, formats and ownership in services — or with custom scalars.



Mask errors

Send generic messages for unexpected errors; log the details with a request ID.



Timeouts & limits

Overall execution timeouts, max request body size, max tokens in a document to stop parser abuse.



Disable what you don't use

Introspection and field suggestions in private APIs, GET mutations (never allow), array batching.

CHECKLIST

A production security checklist

- 01** Depth limit as a validation rule
- 02** Cost analysis with per-field weights
- 03** Max page size on every list field
- 04** Alias and batch limits
- 05** Cost-based rate limiting per client
- 06** Execution timeout and body-size limit
- 07** Trusted documents for first-party clients
- 08** Authorisation at the object-loading layer
- 09** Per-request DataLoaders only
- 10** Error masking and structured logging
- 11** Introspection and suggestions off where private
- 12** Mutations only via POST; CSRF protection for cookies

CHECKPOINT 12

Test yourself before moving on

Q1. Why is depth limiting alone insufficient?

Q2. How can aliases defeat naive rate limiting?

Q3. What do trusted documents do?

Q4. Where should an authorisation check live in a graph, and why?

Answers are in the speaker notes.

SECTION 13 · ADVANCED

Pagination & schema design

Syntax is easy; good schemas are hard. Learn the patterns that make a graph fast, evolvable and pleasant to use for years.

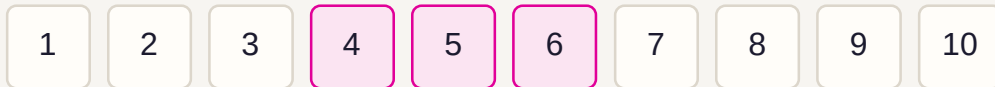
IN THIS SECTION

- 01 Offset vs cursor pagination
- 02 The connection pattern
- 03 Paging through results
- 04 Global object identification
- 05 Naming conventions
- 06 Designing nullability
- 07 Errors as data: result unions
- 08 Evolving without versions
- 09 Breaking vs safe changes
- 10 Anti-patterns

PAGINATION

Offset pagination breaks when data changes; cursors don't

Offset: `books(offset: 20, limit: 10)`



Page 2 = items 4–6. Then a new item is inserted at the top...



Page 3 (positions 7–9) now starts with item 6 again: a duplicate. A delete would skip an item instead. Deep offsets also scan and discard rows.

Cursor: `books(first: 10, after: "c20")`

A **cursor** is an opaque pointer to a specific item. “10 after this item” stays correct however many items are inserted or removed before it.

Cursors typically encode a sort key and a unique tie-breaker (e.g. `created_at + id`) so the database can seek directly with an index.

Use offsets for small, stable, jump-to-page lists (admin tables). **Use cursors** for feeds, infinite scroll and large or changing data.

CONNECTIONS

The connection pattern, as defined by the Relay specification

```
type Query {  
  books(first: Int, after: String,  
    last: Int, before: String): BookConnection!  
}  
type BookConnection {  
  edges: [BookEdge!]!  
  pageInfo: PageInfo!  
  totalCount: Int  
}  
type BookEdge {  
  cursor: String!  
  node: Book!  
}  
type PageInfo {  
  hasNextPage: Boolean!  
  hasPreviousPage: Boolean!  
  startCursor: String  
  endCursor: String  
}
```

1

1. Standard arguments

first/after forward; last/before backward.

2

2. The connection

Wraps a page; room for extras.

3

3. The edge

A node plus its position (cursor).

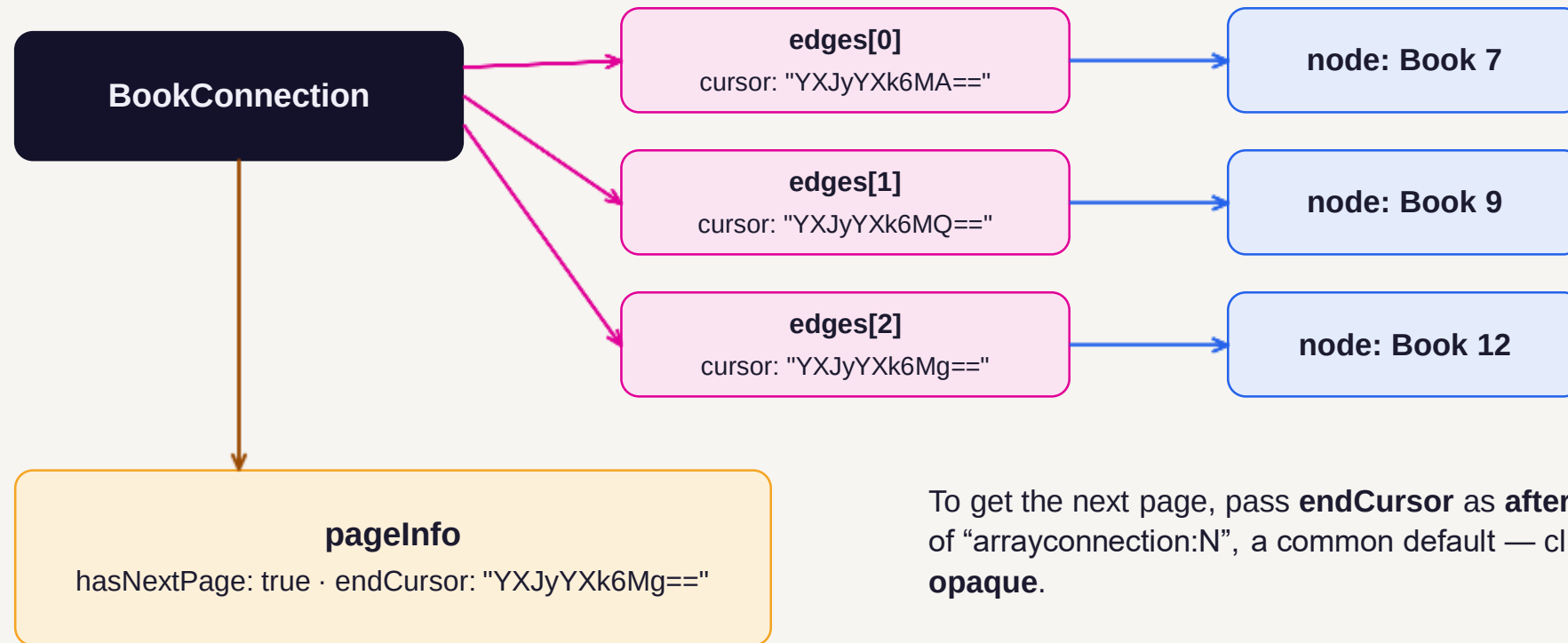
4

4. Page metadata

Is there more, and where to continue.

CONNECTIONS

Anatomy of one page of results



To get the next page, pass **endCursor** as **after**. The cursors shown are base64 of “arrayconnection:N”, a common default — clients must treat cursors as **opaque**.

CONNECTIONS

Paging forward through a connection

```
query Books($after: String) {  
  books(first: 3, after: $after) {  
    edges {  
      cursor  
      node { id title }  
    }  
    pageInfo {  
      hasNextPage  
      endCursor  
    }  
  }  
}
```

- 01** Request 1: { "after": null } → first 3 books.
- 02** Read **pageInfo.endCursor**; if **hasNextPage** is true, send it as **\$after**.
- 03** Repeat until **hasNextPage** is false.
- 04** Many schemas also expose **nodes: [Book!]!** directly on the connection when edge cursors aren't needed.

Enforce a maximum for **first** (e.g. 100) on the server. Reject larger values instead of silently clamping them, so clients notice.

GLOBAL IDS

Global object identification: fetch anything by one ID

```
interface Node {  
  id: ID!      # globally unique  
}  
  
type Book implements Node { id: ID!  title: String! }  
type Author implements Node { id: ID! name: String! }  
  
type Query {  
  node(id: ID!): Node  
  nodes(ids: [ID!]!): [Node]!  
}  
  
# query { node(id: "Qm9vazo3") { ... on Book { title } } }
```

- 01** Global IDs encode the type and the local key — commonly **base64("Book:7")**.
- 02** Clients can refetch **any** object from its ID alone — vital for caches and Relay.
- 03** Keep IDs opaque; clients must never parse them.

CONVENTIONS

Naming conventions the ecosystem expects

Element	Convention	Example
Types, interfaces, unions, enums, inputs	PascalCase	BookReview, SearchResult
Fields and arguments	camelCase	createdAt, first
Enum values	SCREAMING_SNAKE_CASE	PUBLISHED_DESC
Input types	Suffix Input	AddReviewInput
Mutation payloads	Suffix Payload	AddReviewPayload
Connections / edges	Suffix Connection / Edge	BookConnection
Mutations	verbNoun	cancelOrder
Booleans	is / has / can prefixes	isAvailable, hasNextPage

NULLABILITY

A practical nullability policy



Make non-null

IDs, fields computed from the object itself (**title**), lists (**[T!]!**), connection wrappers, **userErrors**.



Keep nullable

Fields from other services or slow sources, lookups by ID (**book(id)**: it may not exist), fields that may be hidden by permissions.

Asymmetry: for outputs, adding **!** is safe and removing it breaks clients. For inputs it is the reverse: making an argument non-null breaks callers.

Recent working-group proposals (semantic nullability, error-aware clients) aim to reduce this tension. Watch for them, but design with today's rules.

Result unions: make every outcome part of the schema

```
union AddReviewResult =
  AddReviewSuccess | InvalidStars | AlreadyReviewed

type AddReviewSuccess { review: Review! }
type InvalidStars { min: Int! max: Int! message: String! }
type AlreadyReviewed { existing: Review! }

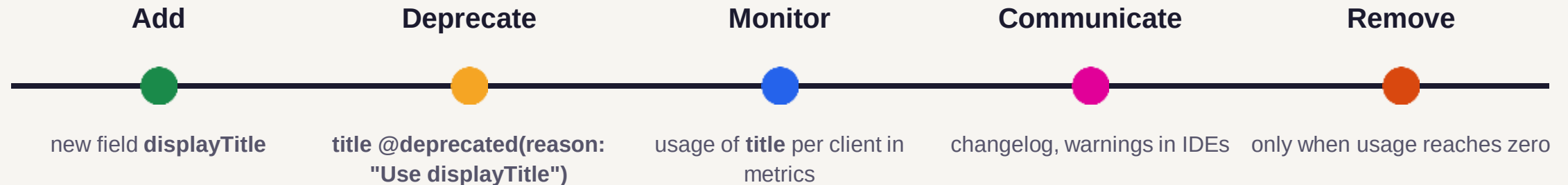
mutation {
  addReview(input: { bookId: "7", stars: 9 }) {
    __typename
    ... on AddReviewSuccess { review { id } }
    ... on InvalidStars { min max }
    ... on AlreadyReviewed { existing { id } }
  }
}
```

- 01** Each failure is a **typed object** with its own useful fields.
- 02** Clients switch on **__typename**; codegen gives exhaustive types.
- 03** Trade-off vs **userErrors**: more precise, more verbose. Adding a member can surprise old clients — give them a fallback branch.

EVOLUTION

Evolve the schema without versions

GraphQL APIs usually **evolve continuously** instead of shipping */v2*. Clients only receive fields they ask for, so adding is safe; removal follows a deprecation cycle.



Field-level usage metrics (from a registry or your own tracing) turn “can we remove this?” from a guess into a query.

EVOLUTION

Safe changes vs breaking changes

Safe (additive)

Change
Add a type, field or query
Add an optional argument
Add an optional input field
Make an output field non-null
Deprecate anything

Breaking

Change
Remove or rename a field, type or argument
Change a field's type
Make an output field nullable
Add a required argument or input field
Remove an enum value; union member changes may surprise clients

Run a schema diff (for example GraphQL Inspector or your registry's checks) in CI on every pull request, and block unreviewed breaking changes.

DESIGN

Anti-patterns to avoid



Database-shaped schemas

Exposing tables and foreign keys (**author_id**) instead of domain objects and relationships.



Generic mutations

updateBook(data: JSON): no validation, no intent, no useful errors.



JSON scalar escape hatches

A **JSON** field opts out of the type system; tools and clients lose all help.



Unbounded lists

reviews: [Review!]! with 100,000 items. Use connections with limits.



Designing for one screen

Fields like **homePageBanner3** couple the graph to today's UI; model the domain instead.



Premature versioning

Adding **v2** fields everywhere instead of the deprecation workflow.

CHECKPOINT 13

Test yourself before moving on

Q1. What problem does cursor pagination fix that offset pagination has?

Q2. Name the four PageInfo fields in the Relay spec.

Q3. Is making an output field non-null a breaking change?

Q4. What are the steps to remove a field safely?

Answers are in the speaker notes.

SECTION 14 · ADVANCED

Clients & caching

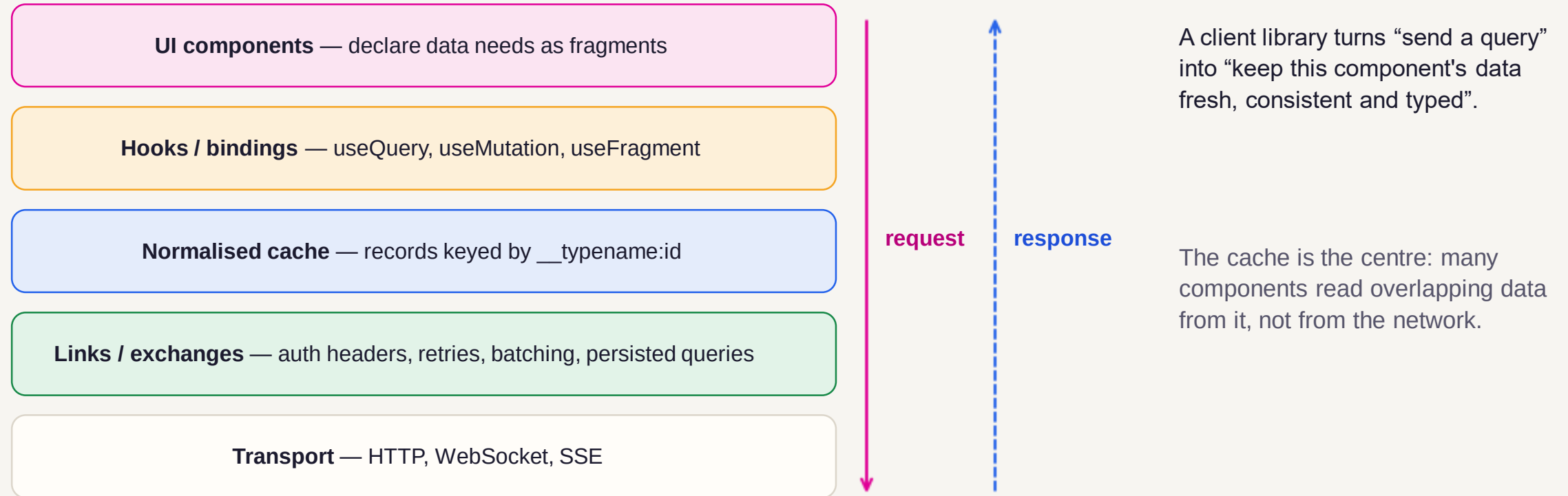
The other half of the contract. Understand what a GraphQL client library adds on top of fetch — and when you need one.

IN THIS SECTION

- 01 What a client does
- 02 Plain fetch is enough to start
- 03 Choosing a client library
- 04 The normalised cache
- 05 Updating the cache after mutations
- 06 Optimistic UI
- 07 Typed operations with code generation
- 08 Fetch policies

CLIENTS

The layers of a GraphQL client



CLIENTS

You can start with nothing but fetch

```
async function gql(query, variables = {}) {
  const res = await fetch("/graphql", {
    method: "POST",
    headers: { "Content-Type": "application/json",
      Accept: "application/graphql-response+json" },
    body: JSON.stringify({ query, variables }),
  })
  const { data, errors } = await res.json()
  if (errors?.length && !data) throw new Error(errors[0].message)
  return { data, errors } // keep partial errors!
}

const { data } = await gql(
  `query($id: ID!) { book(id: $id) { title } }`, { id: "7" })
```

- 01 Perfect for scripts, servers, notebooks and simple pages.
- 02 Python equivalents: **requests.post(url, json={...})**, or **gql / sgqlc** libraries.

Handle **partial** results: data can be present alongside errors. Throwing away data whenever errors exist hides useful results.

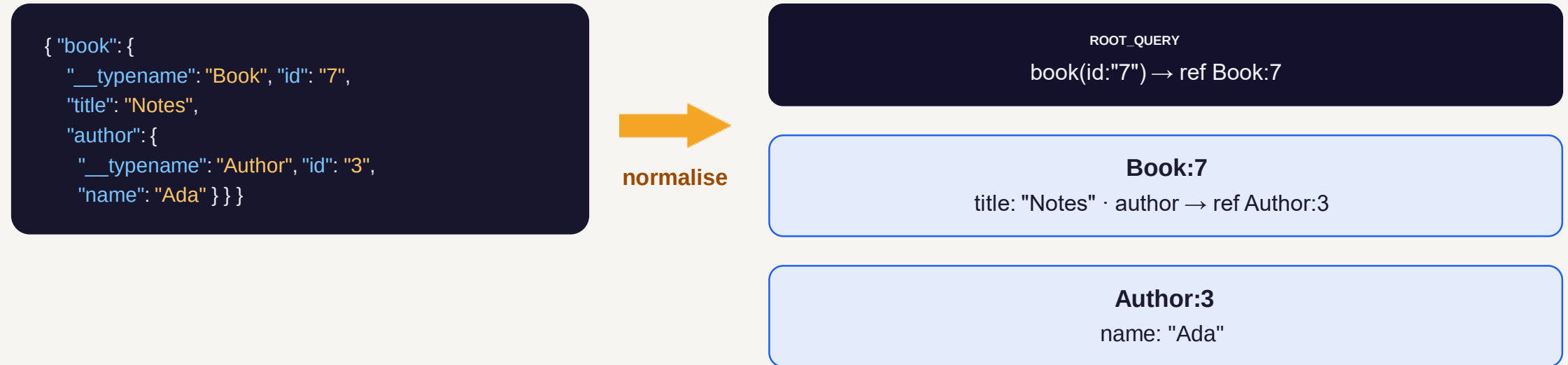
CLIENTS

Choosing a client library

Library	Cache model	Strengths	Consider when
Apollo Client	Normalised	Rich ecosystem, devtools, local state	Large React apps; also iOS and Kotlin clients exist
Relay	Normalised, compiler-driven	Fragment colocation, data masking, pagination built in	Big React apps willing to adopt its conventions
urql	Document cache by default; Graphcache normalised	Small, extensible via exchanges	Lightweight apps; multiple frameworks
TanStack Query + graphql-request	Per-query cache	Simple mental model	Few shared entities; REST and GraphQL mixed

NORMALISED CACHE

A normalised cache stores each object exactly once



Every object with **__typename** + **id** becomes **one record**; nested objects become references. When any query or mutation returns **Author:3** with a new name, every component showing that author updates — because they all read the same record.

CACHE UPDATES

When the cache updates itself — and when you must help



Automatic

A mutation returns an object with `__typename` and `id` — the cached record is merged. Nothing to write.



Manual

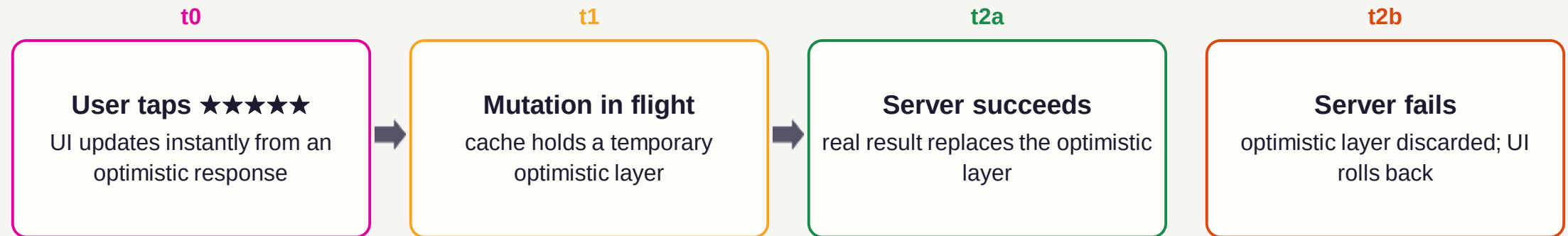
Lists the cache cannot infer: a new review should appear in `book.reviews`; a deleted item should disappear everywhere.

```
// Apollo Client
const [addReview] = useMutation(ADD_REVIEW, {
  update(cache, { data }) {
    const review = data.addReview.review
    if (!review) return
    cache.modify({
      id: cache.identify({ __typename: "Book", id: bookId }),
      fields: {
        reviews: (refs = []) => [
          ...refs,
          cache.writeFragment({ data: review, fragment: REVIEW }),
        ],
      },
    })
  },
})
```

Simplest alternative: `refetchQueries` after the mutation — one more round trip, zero cache code.

OPTIMISTIC UI

Optimistic UI: show the expected result before the server confirms



```
addReview({ variables, optimisticResponse: { addReview: { __typename: "AddReviewPayload",  
  review: { __typename: "Review", id: "temp-1", stars: 5 }, userErrors: [] } } })
```

CODE GENERATION

From schema and operations to fully typed client code

schema + .graphql



codegen



typed hooks

```
// codegen.ts (GraphQL Code Generator)
const config = {
  schema: "http://localhost:4000/graphql",
  documents: ["src/**/*.tsx"],
  generates: { "src/gql/": { preset: "client" } },
}
```

```
import { graphql } from "./gql"

const BookQuery = graphql(`
  query Book($id: ID!) {
    book(id: $id) { title year }
  }
`)

const { data } = useQuery(BookQuery, {
  variables: { id: "7" }, // type-checked
})

data?.book?.year // number | null
```

Types flow from the schema, through your operation's exact selection, into your component. Rename a field in the schema and the compiler shows every broken screen.

FETCH POLICIES

Fetch policies: how a query uses the cache

Policy (Apollo naming)	Reads cache?	Hits network?	Use for
cache-first (default)	Yes — if complete	Only on cache miss	Most screens
cache-and-network	Yes, immediately	Always, then updates	Fast display plus freshness
network-only	No	Always (result is cached)	Data that must be current
no-cache	No	Always (not cached)	One-off, sensitive or huge data
cache-only	Yes	Never	Offline or pre-loaded data

urql calls these request policies with similar names; Relay uses `fetchPolicy` values such as `store-or-network`.

CHECKPOINT 14

Test yourself before moving on

Q1. What two fields does a normalised cache use to identify a record?

Q2. After which mutations must you update the cache manually?

Q3. What happens to an optimistic update if the server fails?

Q4. What does GraphQL code generation take as input?

Answers are in the speaker notes.

SECTION 15 · ADVANCED

Federation & advanced architecture

One graph for the whole organisation, built by many teams. Plus the advanced protocol features that production graphs rely on.

IN THIS SECTION

- 01 Scaling a graph across teams
- 02 Federated architecture
- 03 Entities and @key
- 04 Reference resolvers
- 05 Query planning
- 06 Federation directives
- 07 Composition and registries
- 08 @defer and @stream
- 09 GraphQL over HTTP
- 10 File uploads and observability

SCALING

Three ways to build one graph from many teams



Monolith

One service owns the whole schema.
Simplest to run; becomes a bottleneck when many teams need to change it daily.



Schema stitching

A gateway merges several independent GraphQL schemas and delegates fields.
Flexible; the gateway holds the merge logic.



Federation

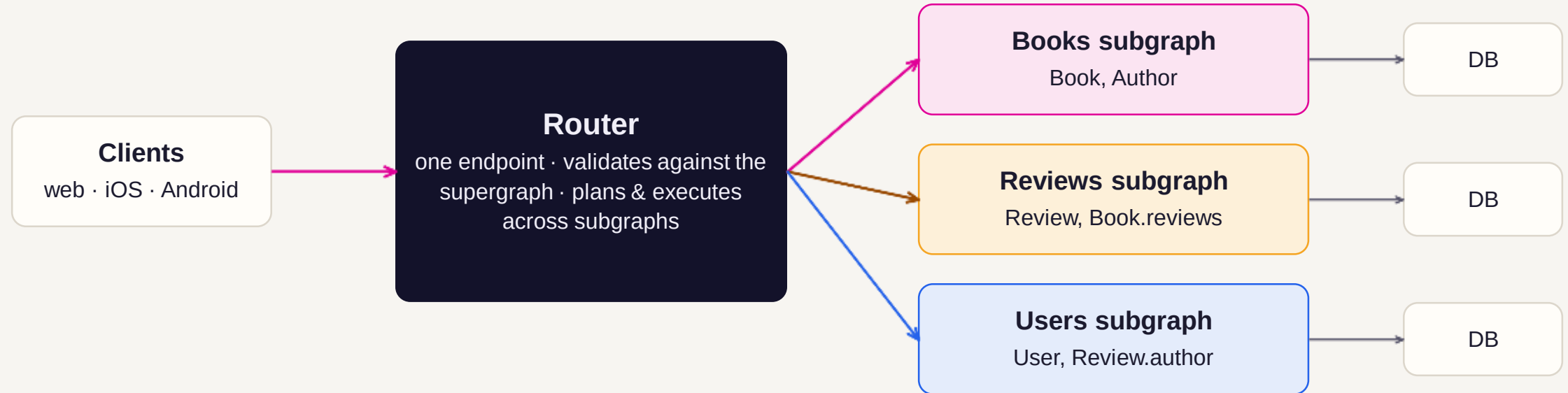
Each service (a **subgraph**) declares how its types relate to others. A router composes a **supergraph** and plans queries across services.

ADVICE

Start with a monolith (modular inside, as in Section 10). Move to federation when team autonomy — not traffic — becomes the constraint.

FEDERATION

A federated graph: one router, many subgraphs



Each team owns a subgraph end to end — schema, resolvers, database, deploys. Clients still see **one** graph; they never know which service answers which field.

ENTITIES

Entities: types that more than one subgraph contributes to

```
books.graphql

# Books subgraph
type Book @key(fields: "id") {
  id: ID!
  title: String!
  author: Author!
}
```

@key makes **Book** an **entity**: any subgraph can refer to it by **id**.

```
reviews.graphql

# Reviews subgraph
type Book @key(fields: "id") {
  id: ID!
  reviews: [Review!]!
}
type Review {
  id: ID!
  stars: Int!
}
```

Reviews adds a **reviews** field to Book without owning Book's other fields.

ENTITIES

Reference resolvers turn a key into an object

```
// Reviews subgraph resolvers
const resolvers = {
  Book: {
    // the router sends { __typename: "Book", id: "7" }
    __resolveReference: (ref) => ({ id: ref.id }),
    reviews: (book, _a, ctx) =>
      ctx.loaders.reviewsByBook.load(book.id),
  },
}
```

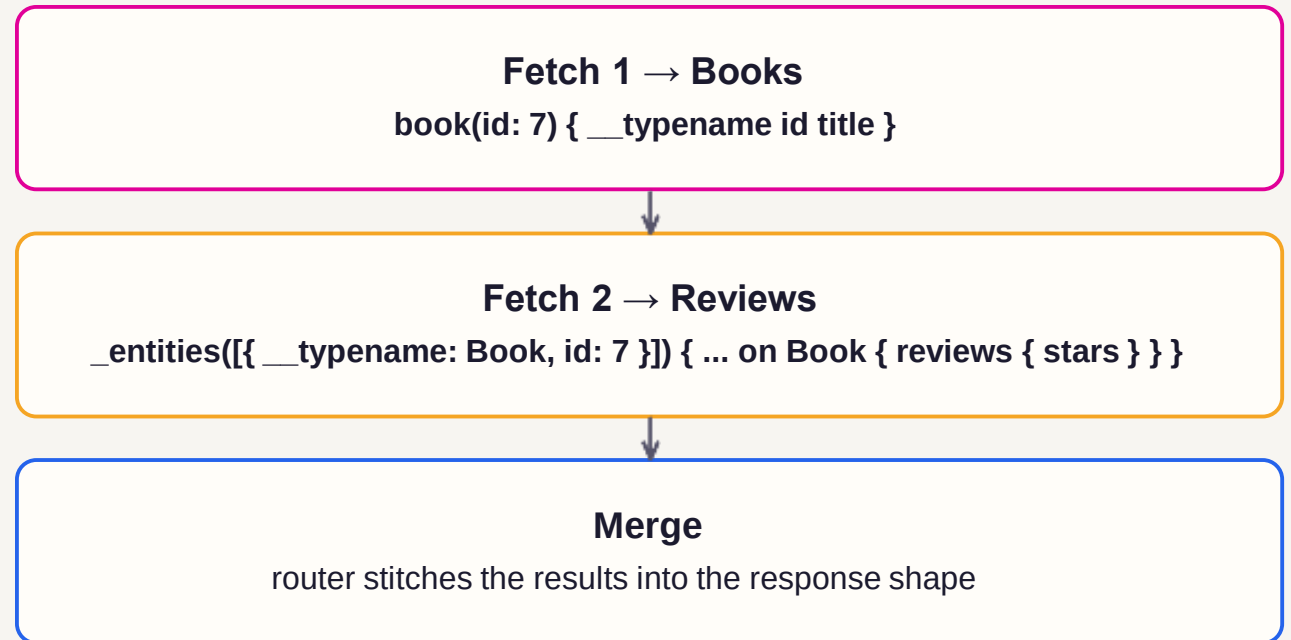
- 01** The router asks a subgraph to hydrate entities from their key fields, via a special **__entities** query.
- 02** **__resolveReference** receives that representation and returns the object — often just the key, if the subgraph only adds fields.
- 03** Batch here too: many representations arrive at once — use DataLoader.

QUERY PLANNING

How the router answers one query from two subgraphs

```
query {  
  book(id: "7") {  
    title      # Books  
    reviews {  # Reviews  
      stars  
    }  
  }  
}
```

The router builds a **query plan**: sequence and parallel fetches, each to one subgraph. Dependent data goes in sequence; independent fetches run in parallel.



FEDERATION

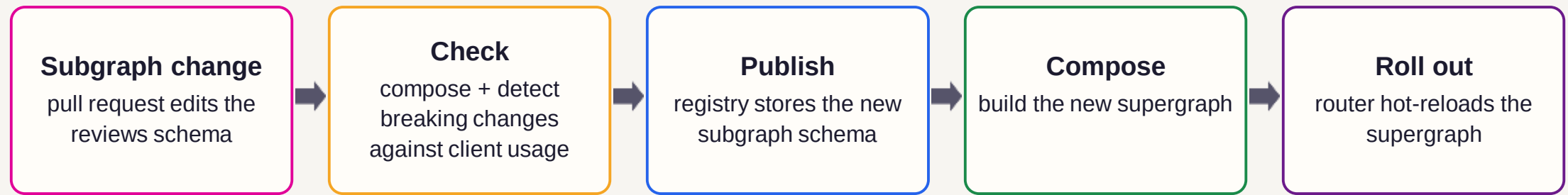
Key federation directives (Apollo Federation 2)

Directive	Meaning
@key(fields: "id")	Declares an entity and the fields that identify it
@shareable	More than one subgraph may resolve this field
@external	The field is defined in another subgraph; referenced here
@requires(fields: "weight")	This field needs other (external) fields fetched first
@provides(fields: "name")	This subgraph can return some external fields on a path
@override(from: "Books")	Migrate ownership of a field from another subgraph
@inaccessible	Hide from the public supergraph, keep for internal composition
@tag(name: "public")	Label elements to build filtered contracts

Other implementations (for example Hive Gateway, WunderGraph Cosmo, GraphQL Mesh, Hot Chocolate Fusion) support this model; an open composite-schemas specification is being developed under the GraphQL Foundation.

COMPOSITION

Composition and a schema registry keep many teams safe



Composition checks that subgraphs agree: shared types have compatible fields, keys resolve, no two subgraphs clash. It happens in CI — before deployment — so a team cannot break the graph for everyone else.

INCREMENTAL DELIVERY

@defer: send the fast part first, stream the slow part

```
query Book {  
  book(id: "7") {  
    title  
    ... @defer(label: "extra") {  
      recommendations { title }  
    }  
  }  
}
```

```
// payload 1 (fast)  
{ "data": { "book": { "title": "Notes" } },  
  "hasNext": true }  
// payload 2 (later)  
{ "incremental": [{ "path": ["book"],  
  "data": { "recommendations": [ ... ] } } ],  
  "hasNext": false }
```

Delivered as a multipart HTTP response or over a subscription transport. **@defer** and **@stream** are still a working-group proposal; payload formats have changed between drafts, so match client and server versions.

INCREMENTAL DELIVERY

@stream: deliver a long list item by item

```
query Feed {  
  feed @stream(initialCount: 5) {  
    id  
    title  
  }  
}
```

- 01** The first **initialCount** items arrive in the initial payload.
- 02** Remaining items follow as incremental payloads as they resolve.
- 03** Great for slow, per-item work (ranking, enrichment); for large lists, pagination is usually still the better design.

HTTP semantics, as standardised by the GraphQL-over-HTTP specification

Situation	application/graphql-response+json	application/json (legacy)
Success, or execution errors with partial data	200	200
Document fails to parse or validate	400 (4xx)	200 typical
Malformed request (bad JSON, no query)	400	400
Mutation sent via GET	405 Method Not Allowed	405
Unsupported Accept type	406	406

Clients should send **Accept: application/graphql-response+json, application/json** and read the body even on 4xx: it still carries the GraphQL **errors**. The spec was maturing through the working-group process at the time of writing.

FILES

File uploads: outside the core spec, three common options



Signed upload URLs

A mutation returns a pre-signed URL; the client uploads directly to object storage, then confirms. Recommended for large files.



Multipart request spec

A community specification: files sent as multipart form parts mapped onto an **Upload** scalar. Needs CSRF protection.



Base64 in a string

Simple but ~33% larger and held entirely in memory. Only for tiny files.

Signed URLs keep large binaries away from your GraphQL servers entirely — they stay fast and simple.

OBSERVABILITY

What to observe in a production graph

Signal	Granularity	Answers
Operation metrics	Per operation name and client version	Which screens are slow or failing?
Resolver tracing	Per field path (OpenTelemetry spans)	Which field made this query slow?
Field usage	Per field, per client	Can we remove this deprecated field?
Errors	By extensions.code and path	Is this a bug, an outage or bad input?
Cost	Calculated cost per operation	Who is consuming our capacity?

Name every operation and send a client name/version header: without them, the metrics above cannot be broken down usefully.

CHECKPOINT 15

Test yourself before moving on

Q1. What makes a type an entity in federation?

Q2. What does `__resolveReference` do?

Q3. When does composition run, and why there?

Q4. What does `@defer` change about the response?

Answers are in the speaker notes.

SECTION 16 · ADVANCED

Testing & operations

A graph is a long-lived contract. Protect it with layered tests and automated checks, and run it with production discipline.

IN THIS SECTION

- 01 A testing pyramid for GraphQL
- 02 Validating client operations in CI
- 03 Schema linting
- 04 Production readiness
- 05 Common incidents and their fixes

TESTING

A testing pyramid for a GraphQL API

Slower, costlier, fewer ↑

End-to-end

few · real client, real server

Contract tests

client operations validated against the schema

Operation tests

execute documents against the real schema

Service unit tests

business rules, authorisation, validation

Static checks

lint, schema diff, composition — every commit

Faster, cheaper, more ↓

Validate every client operation against the schema in CI

```
# .github/workflows/graphql.yml (excerpt)
- name: Validate operations
  run: npx graphql-inspector validate \
    "src/**/*.graphql" schema.graphql

- name: Detect breaking changes
  run: npx graphql-inspector diff \
    origin/main:schema.graphql schema.graphql
```

- 01 **validate** fails if any client document no longer matches the schema.
- 02 **diff** classifies each change as breaking, dangerous or safe.
- 03 With a registry, check against **real client usage** instead: a breaking change to an unused field is harmless.

LINTING

Schema and operation linting catches design drift early

Rule idea	Applies to	Why
Naming conventions (types PascalCase, fields camelCase, enums UPPER)	Schema	Consistency across teams
Require descriptions on types and fields	Schema	Self-documenting API
Require reason on @deprecated	Schema	Migration guidance for clients
Relay connection shape	Schema	Uniform pagination
Named operations only; no unused fragments or variables	Operations	Debuggable, lean documents
No use of deprecated fields	Operations	Clients migrate before removal

graphql-eslint implements many of these rules for both schema files and operations embedded in code.

Production readiness



Limits

Depth, cost, page size, aliases, body size, timeouts — configured and tested.



Performance

DataLoaders per request, parse/validate cache, traces for the slowest operations.



Errors

Masked messages, stable extension codes, request IDs, alerting on error rate.



Change control

Registry, CI checks, deprecation policy, client version headers.

Roll out schema changes behind the usual safety nets: canary deploys for servers, and additive-first changes so rollback never breaks clients.

Common production incidents and their fixes

Symptom	Likely cause	Fix
Database CPU spikes on one screen	N+1 in a new field resolver	DataLoader; query-count assertion in tests
Whole page blank after one service fails	Non-null field on a flaky dependency	Make it nullable; handle null in UI
Sudden traffic surge from one client	Unbounded list or deep query	Cost limits; max page size
Users see each other's data briefly	Shared DataLoader or response cache without user scope	Per-request loaders; scope cache keys
Old mobile app crashes after deploy	Breaking schema change	Schema diff in CI; usage-based checks

SECTION 17 · MASTERY

Capstone: a book-club graph

Bring it all together. Design, query and implement a small but production-shaped API, then check it against a rubric.

IN THIS SECTION

- 01 Requirements
- 02 The schema: types
- 03 The schema: operations
- 04 A page query
- 05 A mutation, end to end
- 06 Review rubric

Requirements for the book-club API



Browse

Members browse books with cursor pagination, filtered by genre, and see each book's author and average rating.



Review

Signed-in members add one review per book (1–5 stars) and see validation errors next to the form.



Clubs

Clubs pick a current book; members see the club's reading schedule and discussion threads.



Live

New reviews on a book appear live for readers viewing that book.



Secure

Queries are cost-limited; only club admins change the current book.



Evolvable

Built to change without versions; every type documented.

The schema, part 1: domain types

```
interface Node { id: ID! }
scalar DateTime

enum Genre { FICTION SCIFI HISTORY POETRY }

type Book implements Node {
  id: ID!
  title: String!
  genre: Genre!
  author: Author!
  averageRating: Float
  reviews(first: Int = 10, after: String):
    ReviewConnection!
}

type Author implements Node {
  id: ID! name: String! books: [Book!]!
}
```

```
type Review implements Node {
  id: ID!
  stars: Int!
  body: String
  by: Member!
  createdAt: DateTime!
}

type Member implements Node {
  id: ID! handle: String!
}

type Club implements Node {
  id: ID!
  name: String!
  currentBook: Book
  members(first: Int, after: String):
    MemberConnection!
}
```

The schema, part 2: entry points and payloads

```
type Query {  
  node(id: ID!): Node  
  books(first: Int!, after: String,  
    genre: Genre): BookConnection!  
  club(id: ID!): Club  
}  
  
type Mutation {  
  addReview(input: AddReviewInput!):  
    AddReviewPayload!  
  setCurrentBook(clubId: ID!, bookId: ID!):  
    SetCurrentBookPayload!  
}  
  
type Subscription {  
  reviewAdded(bookId: ID!): Review!  
}
```

```
input AddReviewInput {  
  bookId: ID!  
  stars: Int!  
  body: String  
}  
  
type AddReviewPayload {  
  review: Review  
  book: Book  
  userErrors: [UserError!]!  
}  
  
type UserError {  
  field: [String!]  
  message: String!  
  code: String!  
}
```

A book page, built from fragments and variables

```
query BookPage($id: ID!, $after: String) {  
  node(id: $id) {  
    ... on Book {  
      ...BookHeader  
      reviews(first: 5, after: $after) {  
        edges { node { ...ReviewItem } }  
        pageInfo { hasNextPage endCursor }  
      }  
    }  
  }  
}  
  
fragment BookHeader on Book {  
  title genre averageRating  
  author { name }  
}  
  
fragment ReviewItem on Review {  
  id stars body createdAt  
  by { handle }  
}
```

Concepts applied:

- 01** Global **node** lookup + inline fragment (S07, S13)
- 02** Colocated named fragments (S04)
- 03** Variables for id and cursor (S03)
- 04** Connection paging (S13)
- 05** Ids everywhere for the cache (S14)

CAPSTONE

addReview, end to end

- 01 Schema:** input + payload with **userErrors** (S05).
- 02 Context:** user from token, per-request loaders (S10).
- 03 Service:** stars range, one review per member (S10).
- 04 Write + publish:** insert, then publish **REVIEW:{bookId}** (S06).
- 05 Return:** review and book, with ids, for the cache (S05, S14).
- 06 Client:** optimistic response; **update** appends to the review list (S14).

DON'T FORGET

Security: **addReview** is weighted in the cost model, rate-limited per member, and reachable only from trusted documents in the first-party app (S12).

AND THIS

Performance: **Review.by** and **Book.author** use **DataLoaders**; **averageRating** is stored and updated on write, not computed per request (S11).

Review rubric: score your own design

Area	Question to ask	Section
Purpose	Does each query return exactly what a screen needs, in one trip?	01–02
Types	Are nullability, lists and enums deliberate and documented?	07
Operations	Are all operations named, parameterised and fragment-based?	02–04
Mutations	Specific names, one input, payload with errors?	05
Performance	No N+1; limits on every list?	11
Security	Depth, cost, auth at the data layer, trusted documents?	12
Evolution	Connections, global IDs, deprecation workflow, CI diff?	13, 16

SECTION 18 · MASTERY

Reference & wrap-up

Two cheat sheets to keep beside you, a glossary of advanced terms, trusted places to keep learning, and the ideas that matter most.

IN THIS SECTION

- 01** Cheat sheet: operations
- 02** Cheat sheet: schema (SDL)
- 03** Advanced glossary
- 04** Where to go next
- 05** Ten ideas to remember

CHEAT SHEET

Operations at a glance

```
# operation with variables + default
query Q($id: ID!, $n: Int = 5) {
  alias: field(arg: $id) {    # alias
    sub
    list(first: $n) { a b }
    ...Frag                  # spread
    ... on Type { onlyHere } # inline
    maybe @include(if: $flag)
    hidden @skip(if: $flag)
    __typename
  }
}
fragment Frag on Type { x y }
```

```
mutation M($in: AddInput!) {
  add(input: $in) {
    result { id }
    userErrors { field message }
  }
}

subscription S($id: ID!) {
  changed(id: $id) { id value }
}

# request body
# { "query": "...",
#   "variables": { ... },
#   "operationName": "Q" }
```

CHEAT SHEET

Schema Definition Language at a glance

```
"""Description"""
type Obj implements I & J @dir {
  f: String!          # non-null
  g(arg: Int = 1): [T!]! # args, list
}
interface I { f: String! }
union U = A | B
enum E { ONE TWO @deprecated(reason: "...") }
input In { x: Int! y: String = "d" }
scalar Date @specifiedBy(url: "...")
```

```
schema {
  query: Query
  mutation: Mutation
  subscription: Subscription
}
directive @dir(a: Int) repeatable
  on OBJECT | FIELD_DEFINITION
extend type Obj { h: Int }

# wrappers: T  T!  [T]  [T!]  [T]!  [T!]!
```

GLOSSARY

Advanced terms from the later sections

Term	Meaning
AST	Abstract syntax tree: the parsed form of a document
Null propagation	A null in a non-null position nulls the nearest nullable ancestor
DataLoader	Per-request batching and memoisation utility that removes N+1 queries
Connection	Relay pagination pattern: edges, nodes, cursors, pageInfo
Trusted documents	Server executes only pre-registered operations, by hash
Cost analysis	Estimating a query's work before execution to limit or bill it
Entity	A federated type identified by @key, contributed to by several subgraphs
Supergraph / router	The composed schema of all subgraphs, and the service that plans queries over it
Incremental delivery	@defer / @stream: a response delivered as several payloads

KEEP LEARNING

Where to go next

Resource	What you'll find
GraphQL specification — spec.graphql.org	The authoritative language definition; readable and precise
graphql.org/learn	Official introductions and best-practice guides
GraphQL-over-HTTP spec (graphql/graphql-over-http on GitHub)	Transport rules, status codes, media types
Relay documentation — relay.dev	Connections, global IDs, fragment colocation in depth
Your server library's docs	Context, plugins, limits, subscriptions for your stack
Federation docs (Apollo, Hive, Cosmo...)	Entities, composition, query planning

Best practice: read the spec's sections on Validation and Execution once. After this tutorial, you'll find them surprisingly readable — and they settle every argument.

SUMMARY

Ten ideas to remember

- 01 A query language lets the **client** declare the data shape it needs.
- 02 The response **mirrors** the query tree.
- 03 The **schema** is the contract; nothing exists outside it.
- 04 Use **variables** for every dynamic value.
- 05 **Fragments** colocate data needs with UI.
- 06 Mutation root fields run **serially**; queries may run in parallel.
- 07 **Nullability** is a promise — and null propagates.
- 08 Resolvers are small; **DataLoader** removes N+1.
- 09 A public graph needs **cost limits** and data-layer authorisation.
- 10 **Evolve** with additions and deprecations, not versions.

END OF TUTORIAL

Ask for exactly what you need.

That sentence is the whole idea of an API query language. Everything else in this tutorial — types, execution, limits, federation — exists to make that promise safe, fast and durable.